

SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis

Nika Mansouri Ghiasi¹ Talu Güloglu¹ Harun Mustafa¹ Can Firtina^{1,2}
 Konstantina Koliogeorgi¹ Konstantinos Kanellopoulos¹ Haiyu Mao³ Rakesh Nadig¹
 Mohammad Sadrosadati¹ Jisung Park⁴ Onur Mutlu¹

¹ETH Zürich ²University of Maryland ³King's College London ⁴POSTECH

Genome sequence analysis, which examines the DNA sequences of organisms, drives advances in many critical medical and biotechnological fields. Given its importance and the exponentially growing volumes of genomic sequence data, there are extensive efforts to accelerate genome sequence analysis. In this work, we demonstrate a major bottleneck that greatly limits and diminishes the benefits of state-of-the-art genome sequence analysis accelerators: the data preparation bottleneck, where genomic sequence data is stored in compressed form and needs to be first decompressed and formatted before an accelerator can operate on it. To mitigate this bottleneck, we propose **SAGe**, an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic sequence data. The key challenge is to improve data preparation performance while maintaining high compression ratios (comparable to genomic-specific compression algorithms) at low hardware cost. We address this challenge by leveraging key properties of genomic datasets to co-design (i) a lossless (de)compression algorithm, (ii) hardware that decompresses data with lightweight operations and efficient streaming accesses, (iii) storage data layout, and (iv) interface commands to access data. **SAGe** is highly versatile, as it supports datasets from different sequencing technologies and species. Due to its lightweight design, **SAGe** can be seamlessly integrated with a broad range of hardware accelerators for genome sequence analysis to mitigate their data preparation bottlenecks. Our results demonstrate that **SAGe** improves the average end-to-end performance and energy efficiency by $3.0\times$ – $32.1\times$ and $13.0\times$ – $34.0\times$, respectively, compared to when the accelerators rely on state-of-the-art software and hardware decompression tools.

1. Introduction

Genome sequence analysis plays an important role in many fields, such as personalized medicine [1–8], tracing outbreaks of communicable diseases [9–14], ensuring food safety through pathogen monitoring [15,16], agriculture [17–19], scientific discovery [20–22], biodiversity conservation [23,24], evolutionary biology [25–27], and antimicrobial resistance surveillance [28–30]. To analyze genomic information computationally, an organism's DNA sample¹ undergoes a process called *sequencing*, which converts the information from DNA molecules to digital data. Current sequencing technologies cannot sequence long DNA molecules end-to-end. Instead, state-of-the-art sequencers generate randomly- and redundantly-sampled

¹Even if an organism's genome is RNA-based, it is typically converted to DNA before sequencing [31,32].

smaller and inexact DNA fragments, called *reads*. Sets of genomic reads (called *read sets*) are then used in genome analysis. The importance of genome sequence analysis, along with the rapid improvement in sequencing technologies (i.e., reduced costs and increased throughput [33]), has led to the rapidly increasing adoption of genome analysis in recent years [1–3,6–9,14] and continuous exponential growth in genomic data generation [34–37] (far outpacing Moore's Law [38]).

It is common practice to store genomic sequence data in compressed forms [33,39–42] because storing uncompressed genomic data is impractical due to its massive size. In fact, due to the importance of storing genomic sequence data in a space-efficient manner, there exist many compression techniques (e.g., [43–59]) specialized for genomic sequence data to achieve significantly higher compression ratios than state-of-the-art general-purpose compression methods (e.g., [60–75]).

Due to the challenges of analyzing massive volumes of genomic data, and its pivotal importance in many critical fields, there are extensive efforts to accelerate genome sequence analysis [76–78]. These efforts focus on alleviating the computational overheads of genome sequence analysis via *algorithmic optimizations* (e.g., [79–87]) or *hardware accelerators* (e.g., [77,78,88–133]), and/or reducing its data movement overheads via *near-data processing* (e.g., in main memory [96–102,133–144] or storage [89,141,145–149]). Unfortunately, all these acceleration efforts are limited greatly by the data preparation bottleneck (which no prior work has fully alleviated), as we describe next.

Data Preparation Bottleneck in Genome Sequence Analysis. We empirically demonstrate that the benefits of prior works on accelerating genome sequence analysis are greatly diminished due to what we call the *data preparation bottleneck*, where compressed genomic sequence data needs to be first decompressed and formatted before it can be analyzed. For example, Fig. 1 shows the execution timeline of data preparation and genome analysis for a real-world genomic dataset (§7) in three different configurations. The evaluated analysis task is read mapping, a fundamental process in genomics (§2.1). The configurations are (i) **Baseline**: a state-of-the-art software analysis tool [81] with a state-of-the-art software genomic decompressor for data preparation [43]; (ii) **Acc. Analysis**: a state-of-the-art hardware-accelerated analysis tool [150] with the same data preparation as Baseline; (iii) **Acc. Analysis w/ Ideal Prep.**: the same accelerated analysis with *ideal* preparation, where preparation is overlapped with analysis. For all configurations, preparation and analysis operate in a pipelined

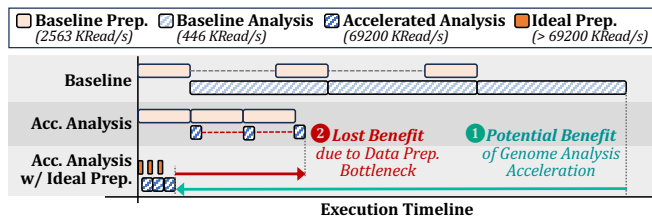


Figure 1: Effect of data preparation (i.e., decompressing and formatting genomic sequence data before analysis) on genome analysis performance.

manner and in batches (i.e., when decompressing batch $#i$, the mapper analyzes batch $#i - 1$).² We observe that ① hardware acceleration of genome analysis can potentially offer substantial performance benefits; ② however, as analysis gets faster, data preparation emerges as a critical bottleneck that hinders the full realization of these benefits. Our motivational study (§3) further demonstrates this bottleneck across various real-world scenarios.

Requirements for Mitigating the Data Preparation Bottleneck. To fully benefit from the extensive efforts on accelerating genome sequence analysis, without resorting to storing massive volumes of genomic sequence data uncompressed, there is a critical need for a design that effectively mitigates the data preparation bottleneck while meeting three key requirements. First, it should achieve *high performance and energy efficiency*. Second, it should maintain *high compression ratios*, comparable to state-of-the-art genomics-specific compression techniques. Third, it must be *lightweight* for seamless integration with a broad range of genome analysis systems, including general-purpose systems, specialized accelerators, portable genomics systems, and near-data processing (NDP) architectures.

Challenges. Meeting all three requirements simultaneously is challenging. Some prior works (e.g., [54,151–158]) accelerate certain computation kernels that are widely used in many state-of-the-art genomic compressors (e.g., [44,48,50,54,58]). Despite their benefits, these approaches require expensive computational units, large buffers, or large DRAM bandwidth or capacity (e.g., due to many random accesses to large data structures for matching patterns). Therefore, none of these approaches is suitable for seamless integration with genome analysis accelerators, particularly when targeting integration into resource-constrained environments, such as various NDP accelerators (e.g., [89,96–102,133–149,159]) or portable genomics devices (e.g., [160–166]), which play critical roles in genomics.

Our goal in this work is to improve the performance and energy efficiency of genome sequence analysis by mitigating the data preparation bottleneck, while maintaining high compression ratios and ensuring a lightweight design. To this end, we propose **SAGE**, an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic sequence data. SAGE is based on the **key insight** that the information encoded in genomic data follows specific trends, shaped by factors such as sequencing technology (e.g., error rates and read lengths) and common genetic phenomena (e.g., typical spatial distributions of genetic

variations within genomes). By carefully exploiting these characteristics to synergistically co-design algorithms and hardware, SAGE achieves high compression ratios, comparable to state-of-the-art genomic compressors, while enabling low decompression latencies, using only lightweight hardware and efficient streaming accesses.

Key Mechanism. Leveraging our rigorous analysis of genomic datasets’ properties, we propose an efficient and synergistic co-design that comprises four aspects. First, SAGE stores information about reads in a genomic read set in array structures that, during decompression, can be interpreted by efficient accesses and lightweight operations. To maintain high compression ratios using *only* these lightweight structures, SAGE’s lossless compression algorithm optimizes the encoding of these structures based on the properties of *each genomic dataset* to accommodate datasets sequenced with different sequencing technologies and from different species. Examples of these dataset properties include sequencing error rates, read lengths, and spatial distributions of genetic variations. Second, we design lightweight hardware units to efficiently decompress SAGE’s data structures. SAGE’s low-cost and lightweight design enables seamless integration with a wide range of genome sequence analysis accelerators, without competing for resources (e.g., memory bandwidth and capacity) required by the genome sequence analysis tasks. Third, we design an efficient data layout to leverage the storage system’s full bandwidth when accessing compressed genomic data. Fourth, we design specialized interface commands that are exposed to genomics applications to access data and communicate with SAGE’s hardware to decompress the data to the desired format.

Key Results. We evaluate the end-to-end performance of various genome analysis applications, where execution includes *both* data preparation and genome analysis. For data preparation, we compare SAGE to the following: (i) pigz [167], a widely-used general-purpose compressor;³ (ii) Spring [43] and NanoSpring [48], state-of-the-art genomics-specific compressors for short and long reads, respectively; and (iii) hardware-accelerated Spring and NanoSpring (see §7 for details). For genome analysis, we evaluate: (i) GEM [150], a state-of-the-art genome analysis accelerator, and (ii) GenStore [145], a state-of-the-art NDP genome analysis accelerator inside the resource-constrained environment of an SSD. We show that when each configuration is integrated with GEM, SAGE improves performance by an average of 12.3 $\times$, 3.9 $\times$, and 3.0 $\times$, and energy efficiency by 34.0 $\times$, 16.9 $\times$, and 13.0 $\times$ over pigz, (Nano)Spring, and hardware-accelerated (Nano)Spring, respectively. When each configuration is integrated with GenStore, SAGE provides 32.1 $\times$, 10.4 $\times$, and 7.8 $\times$ average speedup over pigz, (Nano)Spring, and hardware-accelerated (Nano)Spring, respectively, at a very low area cost of 0.7% of the three cores [169] in an SSD controller [170]. SAGE provides these benefits while achieving significantly larger compression ra-

³We exclude other general-purpose compressors from our performance analysis since, as detailed in §2.2, they deliver substantially lower compression ratios than genomic compressors, and thus do not align with the field’s move toward genomic compressors [52,168]. We include pigz [167], however, since it remains a common comparison baseline in genomic compression studies.

²Decompressed data batches are directly fed to the analysis stage.

tios than pigz (2.9× larger, on average) and comparable ratios to (Nano)Spring (with an average reduction of only 4.6%).

This work makes the following **key contributions**:

- We demonstrate that the *data preparation bottleneck* can greatly limit the potential performance and efficiency benefits of genome sequence analysis hardware accelerators.
- We propose **SAGE**, an algorithm-system co-design for highly-compressed storage and high-performance access of genomic data, to mitigate the data preparation bottleneck, an important bottleneck that is not assessed or handled by prior works on genome analysis acceleration.
- We leverage properties of genomic data to co-design SAGE's algorithm and architecture, such that highly-compressed data can be efficiently interpreted by lightweight hardware and rapidly prepared for analysis.
- SAGE's key contribution lies in the synergistic and genomic-data-aware co-design of algorithms, lightweight hardware, data layout, and a specialized software interface. This synergistic co-design is the key enabler of SAGE's lightweight, high-performance, and energy-efficient design.
- We demonstrate that SAGE significantly improves the end-to-end performance and energy efficiency of state-of-the-art genome sequence analysis hardware accelerators, compared to when the accelerators rely on state-of-the-art software and hardware decompression tools.

2. Background

2.1. Genomic Workflow

Given a genomic sample, a typical workflow consists of three key steps [33,76,77,171], as shown in Fig. 2. The first step, *sequencing* (❶), converts DNA into digital signals. Sequencers cannot read many organisms' entire genomes as complete, error-free sequences. Instead, they produce many shorter overlapping sequences called *reads* that are randomly sampled from different locations in the genome. The average number of reads per location is called the *sequencing depth*. Deeper sequencing provides more information per location, which allows later analysis steps to better distinguish between biological signals and sequencing errors. Sequencers are distinguished by the characteristics of their reads. Examples include sequencers that produce short reads (75 to 300 characters) with high accuracy (~99.9%) [172–175] and long reads (typically 500 to 25k characters, and up to ~2.2M characters [176–178]) with intermediate accuracy (~99%) [175,178–181].

Second, *basecalling* (❷) [77,90,92,182–186], converts the sequencer's raw signals to strings. Each DNA read is converted

to: (i) the DNA encoded using an alphabet representing the nucleotides (also called base pairs) A, C, G, T (along with N to represent unknowns), (ii) a quality score [187] for each nucleotide, encoding its probability of being incorrect using an ASCII character, and (iii) a header. The strings from all reads in a sample (i.e., a read set) are then written to a file.⁴ The file is then typically stored compressed (§2.2) for future analysis.

The third step is *genome sequence analysis* (❸) on a collection of read sets. Since read sets are typically stored compressed [33,39–42], they need to be prepared (i.e., decompressed and formatted) before analysis. A typical analysis workflow quantifies mismatches between the reads and a reference genome. This typically starts with the computationally-expensive *read mapping* process [76,77,81,171,189–191], which finds the potential matching locations of reads in the reference genome. The result of read mapping can then be fed to various downstream analysis tasks (e.g., variant calling [77,192–196], genome assembly [77,103,197–201], and taxonomic classification [134,202–222]). Analysis workflows typically access all base pairs, but only a small fraction of the quality scores. This is because read mapping, while using all base pairs, typically ignores quality scores [48,81,223–225]. The subsequent steps (e.g., variant calling) only need quality scores from the positions surrounding mismatches [193–196] to distinguish true biological variation from sequencing errors.

The analysis step often begins with *existing, already sequenced and basecalled read sets* because, in many cases, a single read set needs to be analyzed *many times* and at *different times* [33,226–229]. For example, some applications (e.g., measuring population genetic diversity [226,230–233]) require analyzing a read set with many reference genomes at different times. Other applications require repeating the analysis many times (e.g., with updated or personalized references [33,226–228,234–237]) to improve accuracy. Due to the criticality of the analysis step and the challenges of analyzing large amounts of sequencing reads on conventional systems, a large body of works (e.g., [76–80,86–140,142–148,159,183,238–255]) accelerate genome sequence analysis tasks, particularly read mapping (e.g., [76–78,86–88,95–114,117–133,140,142–145,159,250–255]), which is one of the most critical bottlenecks in many analysis applications [33].

2.2. Storing Genomic Sequence Data

Criticality of Genomic Sequence Data. Driven by continuing exponential drops in sequencing costs [256,257] and the pivotal importance of genome sequence analysis, genomic sequence data volumes in public [34,36,37] and private [258,259] repositories are growing by an *order of magnitude* every few years, approaching and expected to exceed the data volumes generated by various major internet media platforms [34,36–38]. While genome analysis also works on other data types (e.g., assembled genomes [260–262], epigenetic markers [263–266], and 3D chromosomal contact maps [267–269]), analyzing sequence data is one of the most critical processes in genomics, with sequence data constituting the largest fraction of data stored and analyzed [270]. For example, sequence data is the

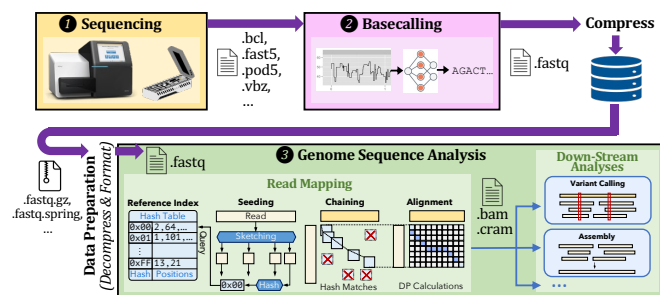


Figure 2: Overview of a typical genomic workflow.

⁴This format is FASTQ [182], the most common read set format [188].

largest class of data maintained at institutions like the National Center for Biotechnology Information and the European Molecular Biology Laboratory [270,271]. Note that many other genomic data types (e.g., assembled genomes, epigenetic markers, and 3D chromosomal contact maps) are also derived from sequence data, so it is critical to store this sequence data to ensure both reproducibility and to enable reanalysis with future improved workflows. For example, while some tasks (e.g., variant calling) can start with already mapped reads, best-practice guidelines dictate that reads should be remapped during the course of a study to ensure the use of appropriate (updated [33,226,227,234–236] or personalized [227,228]) reference genomes and mapping parameters. This is reflected in the fact that, as of October 2025, 75.9% (18 peta DNA bases) of publicly-deposited whole-genome sequencing read sets [34,270] are in unmapped, i.e., FASTQ format [182], and their overall size has been growing at a rate of 42.5% on average per year over the last decade.

Genomic-Specific Compression. Due to the importance of storing genomic data space-efficiently, there exist many compression techniques (e.g., [43–59]) specialized for genomic data. These techniques attain higher compression ratios (e.g., typically from ~ 2 to ~ 40 [43,45,49,51]) than general-purpose compressors (e.g., typically from ~ 2 to ~ 6 [43]), because general-purpose approaches fail to leverage longer-range similarities common in DNA data [39,168]. This large gap in compression ratios exists even with state-of-the-art general-purpose compressors in software (e.g., [60–66]) or hardware (e.g., [67–75]), which is the reason for the large emphasis in genomics to use genomic-specific compressors [52,168]. For example, hardware-based Intel QAT [272] and IBM zEDC [273] compression accelerators lead to $2.6\times$ and $2.7\times$ lower average compression ratios, respectively, than genomic compressors [43,48] on our datasets (§7). Similarly, state-of-the-art general compression algorithms such as xz [274] (a state-of-the-art LZMA-based compressor) and zstd [60], both at their highest levels, lead to $2.13\times$ average (up to $6.7\times$) worse compression ratios than genomic compressors [43,48] on our datasets.

Fig. 3 shows an example of a typical genomic compression technique. Genomic compressors typically represent the DNA bases in each read set with a ① *consensus sequence* and ② the *mismatches* of each read in the read set compared to that consensus [44,47,49,50]. A consensus sequence is an approximation of the organism’s genome, and can either be a user-provided reference [49] or a de-duplicated string derived from the reads, representing the most likely character at each location [44,50]. Note that it is not sufficient to store only the consensus to represent the read set because the consensus alone does not capture the variations in the original reads. Individual reads often contain unique differences, including sequencing errors [166,173] or biological variation [275].

Given a consensus sequence, a lossless encoding of a read’s DNA bases consists of ① the read’s matching position in the consensus sequence (many compressors, e.g., [44,45,48,50], store matching positions based on their order of appearance in the consensus, which enables space-efficient delta-encoding), ② mismatch positions (delta-encoded), ③ mismatch bases and

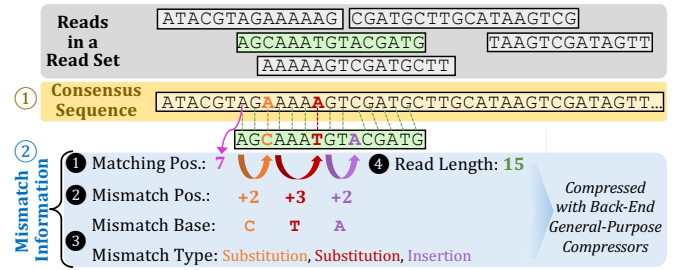


Figure 3: Overview of genomics-specific compression.

types (i.e., substitution, insertion, deletion), and ④ read length (especially for long reads since they have variable lengths). This encoding of the DNA bases is then more compressible using general-purpose compressors [44], which are then used by the state-of-the-art genomic compressors to further compress the mismatch information. Since quality scores do not have the same redundancy patterns as DNA bases, state-of-the-art genomic compressors process them in separate streams and compress them using various techniques (e.g., lossless context models [46,47,51,57,59] or block sorting [43], and lossy quantization [44]). Some tools (e.g., [48,276,277]) discard quality scores altogether since many recent workflows (e.g., [81,202,206,224,278]), particularly those using accurate reads, do not use quality scores.

3. Motivational Analysis

We show the impact of the data preparation bottleneck in §3.1 and discuss challenges of mitigating it in §3.2.

3.1. Data Preparation Bottleneck in Genome Sequence Analysis Accelerators

As shown in Fig. 1, when genome sequence analysis is accelerated, data preparation emerges as a critical bottleneck. We perform experimental studies to understand the impact of this bottleneck when analyzing different real-world read sets.

Methodology. We evaluate the end-to-end performance of a genome analysis application, where execution includes both data preparation and genome analysis. For *genome sequence analysis*, we use a state-of-the-art hardware accelerator [150] for read mapping. For *data preparation*, we consider the following configurations to decompress data into the desired uncompressed format: (i) pigz: A parallel version [167] of gzip, a commonly-used general compressor; (ii) (N)Spr: Spring [43] and NanoSpring [48], state-of-the-art software compressors for short and long reads, respectively; and (iii) Ideal: an idealized compressor with zero decompression time. For our evaluated real-world read sets (§7), Spring and pigz achieve an average compression ratio of 16.9 and 5.4, respectively. We exclude other general-purpose compressors from our performance evaluations since, as detailed in §2.2, they achieve significantly worse compression ratios than genomic compressors, and thus do not align with the substantial need to use genomic compressors [52,168]. We do, however, include pigz (parallel gzip) since it is still widely used as a baseline comparison point in genomic compression research and literature.

We evaluate *end-to-end throughput* based on the throughput of data preparation and genome sequence analysis. We use the

read mapping throughput reported by the original paper [150]. For data preparation, we use a high-end server with 128 physical cores, as detailed in §7. We use the best-performing thread count (i.e., after which adding more threads does not improve performance). I/O operations (reading compressed data), decompression, and read mapping operate in a pipelined manner and in batches, which enables *partial* overlapping of these three steps. Note that the decompressed data batch is streamed directly to the read mapper, without being written to the SSD. Further methodology details are in §7.

Observations. Fig. 4 shows the end-to-end throughput normalized to the configuration with (N)Spr. We observe that data preparation greatly limits end-to-end performance. If the data preparation bottleneck is eliminated, there would be $12.3\times$ and $4.0\times$ average speedup for pigz and (N)Spr.

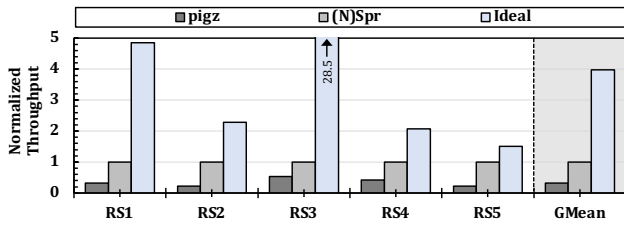


Figure 4: End-to-end throughput for different read sets.

3.2. Challenges and Goal

Requirements. Our observations demonstrate that as accelerators dramatically speed up genome sequence analysis, data preparation emerges as a critical bottleneck in the execution pipeline. Note that storing read sets uncompressed is an inefficient and unsustainable way to mitigate this bottleneck, as they are typically $2\text{--}40\times$ larger. Keeping “hot” data uncompressed is *not* efficient either because genome analysis often involves many read sets that must be processed with the same priority (e.g., a cancer genomics study of whole-genome read sets can range from tens [279] to thousands of samples [280], each around tens to hundreds of gigabytes compressed). As a result, even the “hot” data is too large to be stored efficiently uncompressed. Thus, it is common practice to store read sets compressed even when they are *not* in an archival state, and to support integration with decompression (as seen in widely-used genome analysis software, e.g., [33,81]).

Motivated by our observations, we conclude that there is a critical need to effectively mitigate the data preparation bottleneck while meeting three key requirements: (i) *high performance and energy efficiency*, (ii) *high compression ratios*, comparable to state-of-the-art genomic compressors, and (iii) being *lightweight* to enable seamless integration with a broad range of genome sequence analysis systems (e.g., in FPGAs, GPUs, ASICs, portable devices, or NDP).

Challenges. Meeting all three requirements is challenging. This challenge is even larger in genome analysis systems used in hardware resource-constrained environments. Examples include portable genomics devices (e.g., [160–166]), which are critical to facilitating the wide adoption of genomics, or various NDP accelerators (e.g., [89,96–102,133–149,159]), which lead to large benefits by mitigating data movement overheads of analyzing large-scale genomic data, but are typically im-

plemented in constrained settings (e.g., within the memory system or storage device).

Some works accelerate computational kernels, such as BWT [151–153], FM-index search [154,155], or LZMA [54,157], which are widely used in genomic compressors (e.g., [44,48,50,54,58]). Despite their benefits, these works have two limitations. First, they have high demands for resources like DRAM bandwidth [52,153,156], on-chip buffers [152], DRAM capacity [154], or compute resources [151,155,157]. Efficiently meeting these hardware resource demands is challenging, particularly in resource-constrained environments.⁵ Second, these works accelerate only specific kernels and not the end-to-end genomic decompression process (e.g., reconstructing the full reads from the consensus sequence and mismatches). Thus, they do not fully mitigate the data preparation bottleneck (as shown in §8).

We use an example to elaborate on the limitations of resource-intensive data preparation techniques when integrated with genome analysis systems in resource-constrained environments. Consider NDP genome analysis systems implemented inside the resource-constrained environment of the SSD (e.g., [145–149]). To benefit from such systems, it is essential to efficiently perform data preparation inside the SSD, since moving the data outside the SSD for preparation *completely undermines* the fundamental benefits of in-storage NDP. Unfortunately, performing large amounts of random accesses as needed for data preparation (matching patterns in large data structures during decompression) in the SSD is inefficient. This is due to costly resource contention within the SSD (e.g., in channels and high-latency NAND flash chips [283–287]). Although modern SSDs have an internal low-latency DRAM buffer, its capacity is small (e.g., 4 GB for a 4-TB SSD [170]), with over 95% of it filled with mapping metadata, and its bandwidth is constrained by its *single* channel [288]. Our motivational analysis (§3.1) emphasizes the mismatch between the available resources and data preparation’s resource demands: across our datasets, we observe that the state-of-the-art genomic decompressors [43,48] require random accesses to large amounts of data (up to 26 GB) and with high bandwidth. Consequently, even on a high-end system used in our analysis, with *eight* DRAM channels, 128 cores, and 256 hardware threads, the performance of these genomic decompressors saturates after 32 threads due to insufficient main memory bandwidth.

Our goal is to mitigate the data preparation bottleneck while achieving high performance and energy efficiency, high compression ratios, and a lightweight design. Doing so would unlock the full potential of genome sequence analysis acceleration.

4. SAGe: Overview

We propose **SAGe**, an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic sequence data. SAGe (i) mitigates the data preparation bottleneck, (ii) achieves high compression ratios, and (iii) is lightweight for seamless integration with a

⁵While some genomic (de)compression algorithms [281,282] do not rely on expensive resources, they achieve poor compression ratios, averaging $5.3\times$ lower than state-of-the-art genomic compressors [43,48] on our datasets.

broad range of genome analysis systems. SAGE is versatile, supporting data from different sequencing technologies and species. SAGE's approach is based on the **key insight** that the information encoded in genomic sequence data follows specific trends, shaped by factors such as sequencing technology (e.g., error rates and read lengths) and common genetic phenomena (e.g., typical spatial distributions of genetic variations within genomes). By carefully taking these trends into account when synergistically co-designing algorithms and hardware, SAGE achieves high compression ratios, comparable to state-of-the-art genomic-specific compressors, while enabling low decompression latency using only lightweight hardware and efficient streaming accesses. SAGE's synergistic co-design is the key enabler of SAGE's lightweight, high-performance, and energy-efficient design.

SAGE's co-design comprises four aspects. First, during compression, SAGE encodes the information of reads in hardware-friendly, lightweight *data structures* and exploits genomic data properties in each read set to losslessly *compress* these structures (§5.1). Second, we design *lightweight hardware units* (§5.2) to efficiently interpret and decompress data. Third, we design an *efficient data layout* (§5.3) to leverage the storage system's full bandwidth when accessing genomic data. Fourth, we design *specialized interface commands* (§5.4) that are exposed to genome analysis applications to access data and communicate with SAGE's hardware to decompress the data to the desired format.

Fig. 5(a) shows a high-level overview of SAGE's data preparation for a genome analysis system. When the genome analysis system requests data using SAGE's interface commands (① in Fig. 5(a)), SAGE starts operating based on its data layout (②). SAGE receives compressed data from the storage device (③), decompresses it into the desired format using SAGE's hardware (④), and feeds it to the genome analysis system (⑤). SAGE's lightweight design enables it to efficiently integrate with a broad range of genome analysis systems. In §6, we demonstrate case studies of three ways SAGE can integrate with different genome analysis systems.

Fig. 5(b) shows the high-level overview of how SAGE handles the compression and storage of genome sequence data. Since compression is not on the critical path of genome sequence analysis, we perform it on the host system. The host compresses genome sequence data based on SAGE's compression scheme (① in Fig. 5(b)). When writing the SAGE-compressed data, SAGE leverages its customized interface commands (②) and data layout to facilitate efficient accesses later, during decompression (③).

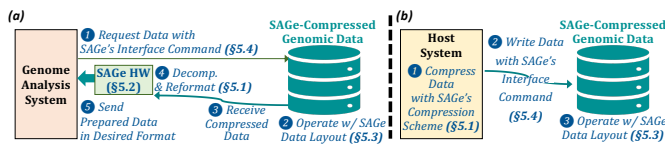


Figure 5: High-level overview of SAGE's (a) data preparation and (b) data compression and storage.

5. SAGE: Detailed Design

5.1. Compression Algorithm and Data Structures

Fig. 6 shows an overview of SAGE's lossless compression technique and its hardware-friendly encodings. SAGE exploits the consensus-based approach (§2.2) widely used in genomic compression (e.g., [44,47,49,50]), storing each read's mismatch information relative to a consensus sequence (①). Similar to existing genomic compressors, SAGE identifies the mismatches during compression by mapping reads to the consensus sequence.⁶ The key difference between SAGE's compression technique over existing genomics-specific compression techniques is the encoding of mismatch information: instead of using expensive backend general-purpose compressors used in genomic-specific compressors (e.g., [43,44,48,50,54,58]), which require costly computational units, large buffers, or large DRAM bandwidth or capacity (e.g., due to many random accesses to large data structures for matching patterns), SAGE (i) stores mismatch information in data structures that can be efficiently decoded by lightweight operations and streaming accesses, and (ii) minimizes their sizes by leveraging read set properties.

To this end, SAGE sequentially stores different components of the mismatch information of all reads in the entire read set contiguously in three arrays. These arrays are dedicated to storing different parts of the reads' mismatch information, i.e., (i) mismatch positions, (ii) mismatch bases and types, and (iii) matching positions. Fig. 6 shows an example of the array used for storing mismatch positions (②). SAGE then adapts the bit width of each array entry and uses a *guide array* to indicate the number of bits used for each array entry (③). This is effective since the mismatch information in genomic datasets tends to follow specific trends, and by using a limited set of bit widths tailored to these trends, SAGE can significantly reduce data size. This approach is lossless since the arrays and guide arrays store a complete and exact representation of all mismatch information, which allows for the perfect reconstruction of each original read's DNA bases from the consensus sequence during decompression. In the rest of this section, we discuss how sequencing technology and genetic phenomena influence the trends in each component of mismatch information, i.e., mismatch positions (§5.1.1), mismatch bases and types (§5.1.2),

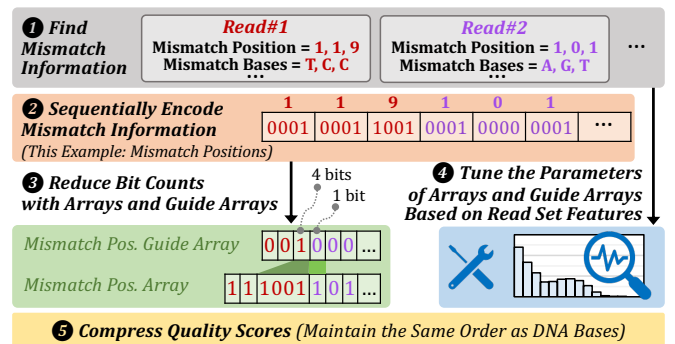


Figure 6: Overview of SAGE's compression mechanism.

⁶Note that this is independent from and does not eliminate the need for read mapping in the later genome analysis stages [227,289].

and matching positions (5.1.3). Since these trends vary for each read set, during compression, SAGE tunes the parameters of arrays and guide arrays for *each read set* (4). The parameters are then encoded at the beginning of the compressed file.

The design of the arrays and guide arrays allows them to be interpreted via streaming accesses. This enables SAGE to reconstruct reads using only lightweight operations during decompression. As a result, SAGE eliminates the need for expensive hardware resources or frequent random accesses.

SAGE also performs lossless compression of quality scores (5). This feature is optional and can be disabled by the user. This is because some accurate sequencers (e.g., [290]) do not report quality scores and print placeholders for format compatibility. Given the accuracy improvements in modern sequencing technologies, many workflows (e.g., [81,202,206,224,278]) no longer use quality scores. While some genomic compressors (e.g., [48,276,277]) do *not* support quality score (de)compression, SAGE supports optional, lossless quality score (de)compression for broad applicability. Since quality scores do not have the same redundancy patterns as DNA bases, SAGE compresses quality scores as a separate data stream from DNA bases, a common practice in genomic-specific compressors [43–45,47,50–52,54,57–59]. SAGE maintains the same order for DNA bases and quality scores during compression. SAGE’s quality score decompression runs on the host CPU because, as shown in §5.1.5, the throughput of SAGE’s quality score decompression is sufficient to avoid becoming a bottleneck in genome analysis pipelines. §5.1.5 provides further details about SAGE’s quality score compression and decompression.

5.1.1. Mismatch Positions and Counts. SAGE stores mismatch positions for reads in a read set sequentially in the Matching Position Array (MMPA) and uses the Matching Position Guide Array (MMPGA) to indicate the number of bits used for each MMPA entry and the number of mismatches in each read. To reduce the sizes of MMPA and MMPGA, SAGE introduces optimizations based on the properties of read sets.

Fig. 7(a) shows the distribution of #bits for the delta-encoded mismatch positions in a representative long read set (RS4 in Table 2). We observe that most delta-encoded mismatch positions need only a few bits to store (**Property 1**). This is due to two main factors that can lead to spatially nearby mismatches. First, genetic mutations tend to cluster in some regions of the genome [291–293]. Second, a regional degradation

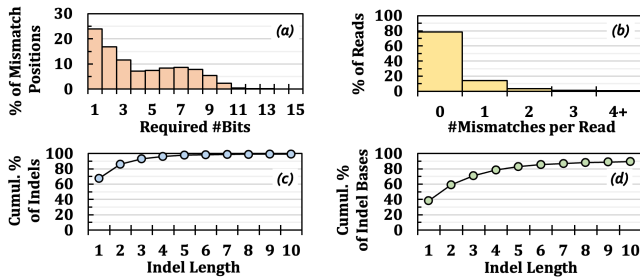


Figure 7: Distribution of (a) #bits needed to store the delta-encoded mismatch positions, and (b) mismatch counts per read. Cumulative distribution of (c) indel block lengths, (d) #bases in indel blocks of different lengths.

in sequencing quality can cause clusters of incorrect bases in the read [294–296]. Based on this, SAGE performs two optimizations. First, during compression of each read set, SAGE tunes the number of distinct bit-counts used for MMPA and the specific values of each bit count. To this end, when finding the mismatch positions of reads, SAGE forms a histogram of bit-counts needed to represent the mismatch positions. SAGE then uses Algorithm 1 to systematically search for the best bit-count boundaries that minimize the total encoded size for that specific read set based on the information encoded in the histogram. While the search for bit-counts is exhaustive, its space is limited, i.e., constrained by a small upper bound on the required bit-widths (typically < 8 in our evaluation). As evaluated in §8.6, this makes the optimization cost very small. Second, SAGE exploits the higher frequency of smaller bit counts to further reduce the guide array size by assigning shorter representations to more common inputs [297]. For example, assuming four distinct bit counts, SAGE uses variable-length prefix codes 0, 10, 110, and 1110 to represent them (instead of 00, 01, 10, and 11). SAGE stores the selected bit counts for the position array and their associated representation in the position guide array in a small *Association Table*.

Algorithm 1 Tuning Bit Counts

Input: Histogram of mismatch position bit counts H , where $|H| \leq 32$, and a tuning convergence threshold ε .

Output: List of bit counts W for encoding mismatch positions.

```

1:  $\ell_{\min} \leftarrow \infty$  ▷ Initialize min. encoding size
2:  $x_0 \leftarrow 0$ 
3: for all  $d \in \{1, \dots, 8\}$  do ▷ Find optimal  $|W|$ .
4:    $\ell_{\text{last}} \leftarrow \ell_{\min}$  ▷ Best result from previous  $d$  values
5:   for all  $(x_1, \dots, x_d) \in |H|^d$  s.t.  $x_0 < x_1 < \dots < x_d$  do
6:      $\ell \leftarrow$  total length of encoded mismatch positions and guide arrays
       s.t. positions with bit counts  $\in (x_i, x_{i+1}]$  are encoded with  $x_{i+1}$  bits.
7:     if  $\ell < \ell_{\min}$  then
8:        $\ell_{\min} = \ell$ 
9:        $W \leftarrow (x_1, \dots, x_d)$ 
10:    if  $(\ell_{\text{last}} - \ell_{\min}) / \ell_{\min} < \varepsilon$  then
11:      break ▷ Exit loop when  $\ell_{\min}$  converges, typically at  $d < 8$ 
12: return  $W$ 
```

Since the number of mismatches can constitute a large fraction of mismatch information in short read sets, we analyze their properties. Fig. 7(b) shows #mismatches per read for a representative short read set (RS2 in Table 2). As also shown by prior works (e.g., [298]), most short reads have no or few mismatches due to short read sequencing’s low error rates (**Property 2**). We exploit this and use the variable-length encoding described above to assign shorter representations to more common inputs.

Given that *indel blocks* (consecutive inserted or deleted bases) cause many mismatches in long reads, we analyze their properties in a representative read set (RS4 in Table 2). Fig. 7(c) and Fig. 7(d) show the cumulative distributions of indel block lengths and the bases stored for different block lengths, respectively. We observe that (i) most indel blocks are of length one, a common trait in read sets [181,299–302], and (ii) although single-base blocks are more frequent, larger blocks encompass most indel bases (**Property 3**). Based on this, we

implement two optimizations. First, we store the position of the first mismatch and the length of the indel, rather than storing each mismatch position. Second, we use the indel lengths' bit-count distribution to minimize the #bits needed for their encoding. To this end, we apply the same systematic bit-count tuning of Algorithm 1. Given the strong skew towards 1-bit indels [181,299–302], we observe that the best configuration typically consists of two distinct bit counts: one for single-base indels and one for others. Accordingly, after detecting an indel (see §5.1.2), we reserve one bit in MMPGA to indicate whether it is a single-base indel, and otherwise, we dedicate eight bits to encode the indel length, as we observe these bit counts to be suitable fits across our evaluated datasets. In the uncommon case where longer indels are more frequent, SAGE can use Algorithm 1 to find other bit count configurations that minimize the encoded size.

Based on all the optimizations discussed in this section, SAGE tunes the encoding of position arrays and position guide arrays to compress mismatch position information for all reads in a read set (as described in Fig. 6). Given the lightweight structures of the position arrays and guide arrays, SAGE can decompress mismatch position information with simple operations and efficient access patterns.

Example Walkthrough of Mismatch Position Decompression. Fig. 8 shows an example of decompression of mismatch positions, demonstrating the position array MMPA (❶), the position guide array MMPGA (❷), and the small Association Table (❸). Based on the optimizations described in this section, during compression, the bit counts used for representing the MMPA elements are selected, and the associated representations used for these bit counts in the MMPGA are optimized and stored in the Association Table. In the example of this figure, three distinct bit counts are used for mismatch positions stored in the MMPA (i.e., 2, 4, and 8), and these bit count values are represented by 0, 10, and 110 in the MMPGA.

To decompress mismatch positions, SAGE first reads the mismatch count of Read#1 in MMPGA (i.e., 0011). It then scans MMPGA to obtain the bit counts for each three mismatch positions of Read#1 and scans the specified number of bits from MMPA to decode these positions. For example, the first entry in MMPA after the mismatch count is 10. Based on the Association Table, SAGE knows that it needs to read the next 4 bits in the MMPA (i.e., 1110) to decode the mismatch position. If a mismatch is an indel, SAGE checks MMPGA to determine if the indel is longer than one. If so, it reads the next eight bits from MMPA to determine its length. After decoding all mismatch positions of Read#1, SAGE proceeds to the next read.

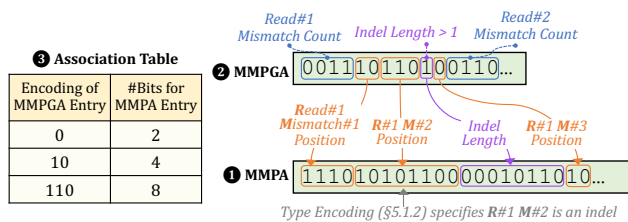


Figure 8: Example of decoding mismatch positions for long reads in SAGE.

5.1.2. Mismatch Bases and Types. SAGE stores mismatch bases/types information of reads in a read set in a Mismatch Base and Type Array (MBTA) and reduces its size with two optimizations based on properties of genomic data. First, in long reads, we observe that a significant fraction of mismatch bases (e.g., up to 80% in our datasets) can originate from *chimeric* reads, which are reads with sequences joined from different regions of the genome due to sequencing/library preparation errors, or structural variations [303] (**Property 4**). Parts of these reads map to different locations in the consensus. Fig. 9 shows a chimeric read with eight mismatches at matching position#1 and nine at position#2. Thus, considering only the top matching position (i.e., position#1 in this example), as in prior works (e.g., [49,50]), results in many mismatches (i.e., eight). Instead of relying on expensive compressors to compress these mismatches, SAGE considers the top N matching positions⁷ for chimeric reads. For example, considering both positions in Fig. 9 (positions #1 and #2) reduces mismatches to three, fewer than the eight mismatches at the top matching position. Since, in chimeric reads, the number of mismatches at individual positions can be substantial, reconstructing reads from multiple positions (and storing these positions) is more efficient than storing many mismatches from only the top position.

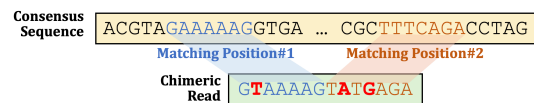


Figure 9: Example of a chimeric read.

Second, since substitutions are the most common mismatch in short reads due to specifics of sequencing technologies [173,304–307] (**Property 5**), we design a new encoding to avoid explicitly storing their type. SAGE compares the mismatching base with the consensus sequence at the corresponding mismatch position. If they differ, the mismatch is a substitution. If they are the same, the mismatch cannot be a substitution and must be an indel. We then use a single bit to distinguish between an insertion and a deletion. This bypasses the need to explicitly store substitution types.

5.1.3. Matching Positions. Since sequencers read each part of the genome several times (to better allow later analysis to distinguish between biological signals and sequencing errors, §2), many reads in a read set match closely in the consensus sequence (**Property 6**). Since each read in a read set can be *mapped independently* [33], it is possible to reorder them based on their matching positions in the consensus [43–46]. Thus, consecutive reads require only a few bits to store their delta-encoded matching positions. Fig. 10 shows this trend by demonstrating the distribution of the number of bits needed to store the delta-encoded matching positions in a representative real human read set (RS2 in Table 2). While a similar pattern of skew towards specific bit counts can be seen in other data types (e.g., images [308]), the possibility to *reorder the reads* enables leveraging this pattern in genomic data more efficiently. Several genomic compression algorithms (e.g., [43–46])

⁷We use $N = 3$ as it led to the best results in our evaluated datasets. SAGE can tune N to other values as well.

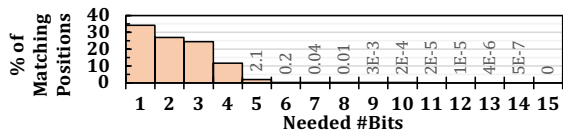


Figure 10: Distribution of #bits needed to store the delta-encoded matching position.

also reorder reads for delta encoding and then apply entropy coding to further compress the matching positions. However, their entropy decoding incurs expensive random lookups to match patterns during decompression. Instead, SAGE uses its hardware-friendly structures to significantly compress the matching positions. To this end, SAGE sequentially encodes the delta-encoded matching positions of reads in a read set in the Matching Position Array (MPA) and the Matching Position Guide Array (MPGA). SAGE then uses the same approach used for reducing the sizes of position arrays and position guide arrays of mismatch positions in §5.1.1 to reduce the sizes of MPA and MPGA.

5.1.4. Corner Cases. Two corner cases require special handling. First, reads containing unidentified bases (N) expand the DNA alphabet to five characters, making 2-bit encoding impossible. Second, some reads include *clips* (i.e., large insertion blocks at the beginning or end) that need reattachment during decompression. Adding indicator bits to every compressed read to specify these cases would significantly increase overhead (particularly for short reads). To avoid this, SAGE marks a read as a *corner case* by encoding a mismatch as position 0. SAGE then uses a single bit in the MBTA to indicate whether a read has a mismatch at position 0, indeed, or is a corner case. By doing so, SAGE efficiently identifies and handles corner cases without impacting standard reads.

5.1.5. Quality Scores. SAGE *losslessly* compresses quality scores. As discussed in §5.1, this feature is optional and can be disabled. SAGE compresses quality scores as a separate data stream from DNA bases, a common practice in genomic compressors (§2). SAGE maintains the same order for DNA bases and quality scores when reordering the reads (§5.1.3).

SAGE’s quality score decompression runs on the host CPU because its throughput is sufficient to avoid becoming a bottleneck in genome analysis pipelines. This is because applications that require quality scores, such as variant calling (performed on reads generated with sequencing techniques that report quality scores), typically only access scores for a small fraction of positions surrounding mismatches (as identified during the earlier read mapping step) [193–196]. For example, across our evaluated read sets, on average only 0.03% (maximum 10.7%)⁸ of quality score blocks (with a block size of 25 MB [312]) are accessed. This ratio is small because very few genomic locations are variable within a population [313,314]. For these quality score blocks, we find that the time to decompress them on the host CPU is significantly shorter than mapping the entire

⁸This ratio approaches upper limits of dissimilarity between an individual’s genome and a reference genome (which determines the fraction of accessed quality scores) typically tolerated in genome analysis. At greater dissimilarity levels, the efficacy of standard genome analysis is often compromised [309–311].

read set using a read mapping accelerator [150]. Therefore, given that reads can be analyzed in batches and in a pipelined manner, the quality score decompression is *not* on the critical path of the pipeline. For our system configuration (§7), quality score decompression on the host CPU would not become a bottleneck for cases where up to 17% of quality scores are accessed. This threshold provides a safe margin, ensuring that quality score decompression does not bottleneck execution in the vast majority of cases.

SAGE’s quality score (de)compression is based on the same software (de)compression used for quality scores in [43] (its lossless mode). Since SAGE’s quality score decompression runs on the host, it can also flexibly adopt other algorithms (e.g., [46–59]).

5.2. Hardware for Data Preparation

5.2.1. Components. Fig. 11 shows the structure of SAGE’s lightweight hardware units. SAGE consists of three components. ① Scan Unit (SU) sequentially scans through input position arrays and position guide arrays to find mismatch information. ② Read Construction Unit (RCU) receives the mismatch information from the SU and reconstructs full reads by plugging the mismatches into the correct positions of the input consensus sequence. ③ Control Unit (CU) coordinates operations between the SU and the RCU.

SAGE enables decompression with streaming accesses, and thus, the SU and the RCU do *not* rely on large buffers, and instead only require small registers. Since SAGE stores mismatches in the order of appearance in each read, decoding is feasible via sequential accesses to the position arrays and position guide arrays. Similarly, since SAGE stores the reads’ positions based on their order in the consensus (similar to other compressors, e.g. [43–46], as explained in detail in §5.1.3), it only sequentially accesses the consensus sequence. The registers used for buffering the position arrays and position guide arrays each consist of eight bits since that is the largest element size used in the arrays in our compression scheme (§5.1). SAGE also uses an eight-bit register since it loads the configuration parameters in eight-bit chunks. SAGE uses a register of size 150 base pairs since this is the largest read size in most short read sequencing datasets [304,305]. For short reads longer than this and for long reads, we reconstruct the reads in 150-base-

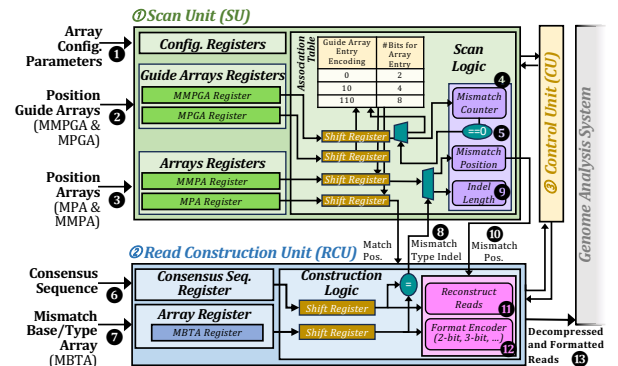


Figure 11: Overview of SAGE’s hardware.

pair chunks.⁹ SAGE directly sends reads to the analysis system. Since reads can be analyzed independently, analysis can start as soon SAGE's output arrives.

5.2.2. Operations. When SAGE receives a request to access a genomic read set, it starts preparing the reads in the read set. The SU reads the tuned configuration parameters of the read set (❶ in Fig. 11). This includes the contents of the Association Table (as shown in Fig. 8). The SU and RCU then start operating concurrently to decode the mismatch information and reconstruct the full reads. The SU decodes each read's matching position and mismatch positions by scanning through the position guide arrays (❷) and position arrays (❸), as described in §5.1. Each time a new mismatch position is decoded, the mismatch count is decremented (❹), and when it reaches zero (❺), the SU reads a new mismatch count (for the next read). Meanwhile, the RCU decodes the mismatch bases and types (as described in §5.1.2) and reconstructs the full reads. The RCU scans through the consensus sequence (❻) and the MBTA (❼) and when it detects an indel mismatch type, it sends a signal (❽) to the SU to read the indel length (❾). After decoding each read's matching position and its mismatch positions, the SU sends them to the RCU (❿). Based on the information received from the SU, the RCU reconstructs each read by applying the mismatches to the corresponding positions of the consensus sequence (⓫), and flexibly formatting the reads (e.g., in 2-bit encoded, 3-bit encoded for reads with N, ASCII, etc.) as requested (⓬). Finally, SAGE sends the decompressed and formatted reads to the genome analysis system (⓭).

5.3. Data Layout

We describe SAGE's efficient data layout, which enables it to leverage the storage system's full bandwidth when accessing and preparing genomic data. We also discuss the feasibility and design efforts required to maintain this layout alongside the existing flash translation layer (FTL). SAGE requires simple changes to the baseline FTL. SAGE FTL designates each block as genomic or non-genomic. The SSD recognizes genomic accesses through SAGE commands (§5.4). For all other data, vendor-specific FTL features remain untouched, so the SSD behaves like a conventional SSD.

When writing a compressed genomic dataset, SAGE uniformly partitions data across the SSD channels. This is enabled by SAGE's sequential access patterns. Each partition of the consensus sequence, along with the compressed mismatch information of the reads matched to that partition, is placed in a separate channel. SAGE writes data in a round-robin fashion across different channels such that the active blocks in different channels have the same page offset. This enables *multi-plane* read operations across all channels and leverages the SSD's *full* bandwidth.

During garbage collection (GC) [284,315–318], we must select victim blocks such that we preserve SAGE's capabilities for multi-plane operations. This requires maintaining the same page offset across all blocks in a parallel unit. Given that genomic read sets do not require partial updates [319,320] and are

accessed sequentially, it is straightforward to realize this efficient GC. We perform GC in a grouped manner and select every block in the parallel unit as a group of victim blocks, which are then sequentially rewritten in the order they were originally written, as indicated by their logical address sequence.

5.4. Interface Commands

SAGE introduces two new interface commands: one to request genomic data in the desired format, and another to write compressed genomic data to the storage system.

SAGE_Read: A specialized read command to (i) access genomic data and (ii) specify the output format (e.g., 2-bit or 1-hot encoding) required by the genome analysis system. The other parameters (e.g., array/guide array parameters) are written at the beginning of each compressed file and are loaded into the SU (§5.2) when accessing genomic data. Upon receiving this command, SAGE operates with its FTL (§5.3).

SAGE_Write: A specialized write command for writing genomic data to SSD and updating its FTL's mapping metadata.

6. Case Studies of SAGE's Hardware Integration

Fig. 12 shows three examples of SAGE's integration with genome analysis systems. In ❶, SAGE's hardware units are connected to the genome analysis system via a PCIe [321] or CXL [322] interface (i.e., similar to how individual accelerators or GPUs connect). In ❷, SAGE's hardware is integrated on the same chip as the genome analysis system (which can be a stand-alone ASIC, or an FPGA, etc.), allowing for tighter integration and saving PCIe or CXL ports, which are scarce and can be contested by other components (e.g., SSDs, GPUs, or other accelerators). This is easily possible due to SAGE's low hardware cost. For example, SAGE's logic units (§8.2) consume only 2.5% of the lookup tables and 0.8% of the flip-flops of a mid-range FPGA [323]. SAGE's lightweight design enables its integration in resource-constrained environments. As an example, in ❸, we demonstrate SAGE's integration with an in-storage NDP genome analysis system (e.g., [145–148]) on the SSD controller. Given that SAGE performs streaming accesses, SAGE's hardware operates on data fetched from NAND flash chips without needing to buffer them in the SSD's low-bandwidth, single-channel, internal DRAM [288]. SAGE performs operations on flash data streams [288] via lightweight double-buffering: one register to hold a chunk of data as computation input, and another to hold the subsequent data chunk arriving from the NAND flash chips. We use two 64-bit registers as they are sufficient to fully utilize the per-channel hardware components and SSD channel bandwidth.

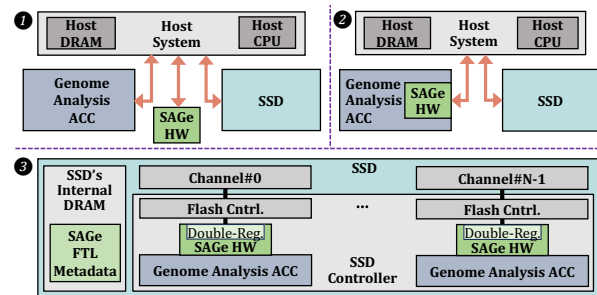


Figure 12: SAGE's integration with genome analysis systems.

⁹As detailed in §6, in the third integration mode in Fig. 12, SAGE requires two additional 64-bit registers for its operations.

7. Methodology

Evaluated Systems. We evaluate the end-to-end performance of various genome analysis systems, where execution includes *both* data preparation and genome analysis. For **data preparation**, we use (i) *pigz*: A parallel version [167] of *gzip*, a commonly-used general compressor. We exclude other general-purpose compressors from our performance evaluations due to reasons detailed in §3.1. We do, however, include *pigz*, i.e., parallel *gzip*, since it is still widely used as a baseline comparison point in genomic compression research and literature [43–48,51–59]; (ii) (N)Spr: Spring [43] and NanoSpring [48], state-of-the-art compressors for short and long reads, respectively. *pigz* and (N)Spr run on a high-end system, an AMD® EPYC® 7742 CPU [324] with 128 physical cores (256 hardware threads). (iii) (N)SprAC: (N)Spr integrated with a BWT-accelerator. There are various accelerators for BWT (e.g., [151–153]). We consider an idealized accelerator that can fully eliminate the BWT execution time from (N)Spr; (iv) *θTimeDec*: an idealized decompressor (with zero decompression time), but inefficient for integration in resource-constrained environments (e.g., in our evaluations, for integration with an in-storage NDP genome sequence analysis system);¹⁰ (v) SAGE_{SW}: SAGE with its decompression in software, running on the host system (i.e., the same high-end, 128-core system used for *pigz* and (N)Spr), to show the benefits of SAGE’s algorithmic optimizations; (vi) SAGE: SAGE’s full implementation, with its decompression in hardware; and (vii) SAGE_{SSD}: SAGE with its hardware implemented in the SSD, to integrate with an NDP genome analysis system on the same chip (mode ③ in Fig. 12). Decompressors other than SAGE_{SSD} connect to the analysis systems using a PCIe interface (① in Fig. 12). We do not explicitly evaluate mode ②, because given sufficient interface bandwidth, mode ② can perform the same as ①. As detailed in §6, ① enables standalone integration, while ② integrates on the same chip, thus saving PCIe slots.

We select Spring [43] and NanoSpring [48] as our genomics-specific software baselines because they provide a well-balanced and particularly suitable trade-off in the key aspects of compression ratio, decompression time, losslessness, and being open access. In contrast, other tools have less favorable characteristics in compression ratio (e.g., [44,45]), decompression time (e.g., [47,49,51]), lossiness (e.g., [44,45]), or do not permit code access and/or modification (e.g. [52,58]).¹¹ As explained earlier, we use *θTimeDec* as a strongly conservative and idealized representation of the closed-source tools.

For **genome sequence analysis**, we integrate all data preparation configurations with a state-of-the-art read mapping accelerator, GEM [150]. To show SAGE’s suitability for resource-

¹⁰As a conservative evaluation, *θTimeDec* can also serve as an idealized representation of the closed-source industry software called ORA [52], recently developed for Illumina [325] short read (de)compression. ORA works similarly to other genomic compressors (§2.2), but uses a different backend compressor for mismatch information, which still suffers from limitations for implementation in resource-constrained environments (detailed in §3.2). We cannot use ORA as our short-read decompression baseline due to its closed-source nature, but its performance would be upper-bounded by *θTimeDec*.

¹¹Being able to modify the code is critical for us since we need to adapt it to pass decompressed data batches to the analysis system (without merging data into a single file and writing it to disk).

constrained environments, we evaluate SAGE’s implementation inside the SSD to integrate with a state-of-the-art NDP genome analysis system, GenStore [145]. GenStore is an in-storage filter (ISF) that filters reads that do not require expensive read mapping directly inside the SSD, sending only the remaining reads to the mapper, thus alleviating the burden of moving and analyzing a large amount of low-reuse data from the rest of the system (i.e., external I/O, main memory, and compute units). The resulting pipeline performs data preparation → ISF → read mapping. The benefits of this pipeline arise from both SAGE’s faster preparation and the ISF operations in [145]. The key to realizing this pipeline is that SAGE is the only data preparation configuration that is lightweight enough for efficient implementation inside the SSD. Without SAGE, ISF would require genomic data to be stored uncompressed, which is inefficient, or to decompress data outside SSD, which undermines the fundamental benefits of NDP.

Datasets. We evaluate real-world short- and long-read datasets. Storage systems handle numerous read sets (e.g., in a medical center [258,259] or in a cohort [258,259,314,326]), so compressing each read set is essential to reduce the overall burden, regardless of their individual sizes. Thus, we analyze read sets of varying sizes, as shown in Table 2.

Performance. We design a simulator that models all components involved during SAGE’s execution, including accessing storage, hardware components (both for SAGE and genomic accelerators [145,150]), the SSD’s internal DRAM (for the NDP accelerator [145]), host operations, and their interfaces. Using this methodology, as also demonstrated in prior works (e.g., [145,147,327]), enables us to flexibly incorporate state-of-the-art system configurations in our analysis. We feed the latency and throughput of each component to this simulator. For the components in **hardware-based** operations, we implement SAGE’s logic units in Verilog and synthesize using Design Compiler [328] at 22 nm [329]. We use Ramulator 1.0 [330–333] to model the SSD’s internal DRAM, and MQSim [334,335] to model the SSD’s internal operations. For the hardware mapper, we use the throughput reported by the original paper [150]. For the **software-based** operations (software decompressors), we measure performance with their best-performing thread counts, on a high-end real system, an AMD® EPYC® 7742 CPU [324] with 128 physical cores, 256 hardware threads, and 1.5-TB DRAM. We perform analysis with both a performance-optimized PCIe SSD [336] and a cost-optimized SATA SSD [337]. Data communication between genome analysis accelerators, SAGE’s hardware units, and the storage system is modeled based on the bandwidth of the interfaces between them, depending on integration mode (Fig. 6). I/O operations, decompression, and genome analysis execute on batches of genomic data in a pipelined manner, which enables partial overlapping of their execution. The synchronization between these stages is modeled via a producer-consumer abstraction.

Prototype Feasibility and Real-System Overheads. While any real-world prototype will introduce overheads not perfectly captured in simulation, we expect that system-level software changes in the I/O stack to have only a negligible impact. The required changes to the file system and the driver are con-

fined to distinguishing genomics file types and supporting two new interface commands (§5.4). Since this adds only simple new logic, we expect it to lead to negligible performance impact. The firmware needs to support SAGE’s L2P mapping and GC (§5.3), which are lighter than the baseline (due to SAGE’s uniform distribution of data across the SSD channels) and are already modeled by MQSim [334].

Area, Power, and Energy. For the accelerator logic units, we use the area and power values obtained from the Design Compiler synthesis [328] of our Verilog HDL implementation of these units in a 22nm technology node [329]. For DRAM, we use the power values of a DDR4 model [338–340]. For the CPU cores, we use power values obtained from AMD® µProf [341]. For the SSD, we use the power values of a Samsung 3D NAND SSD [170]. We calculate the energy of each component based on its idle and dynamic power and its execution time. For end-to-end energy, we obtain the energy of the host processor, DRAM, SAGE’s logic units, and communication between them.

8. Evaluation

8.1. Performance

Fig. 13 shows end-to-end performance, where execution includes *both* data preparation and genome analysis. Speedup is normalized to (N)Spr. We make five key observations. First, SAGE provides significant speedups. On the system with the PCIe (SATA) SSD, SAGE leads to $12.3\times$ ($8.1\times$), $3.9\times$ ($2.7\times$), and $3.0\times$ ($2.1\times$) average speedup over pigz, (N)Spr, and (N)SprAC, respectively. Second, SAGE matches 0TimeDec in performance because SAGE fully hides the decompression overhead in the execution pipeline. As explained in §3, I/O, decompression, and read mapping are pipelined, so overall throughput depends on the slowest stage. 0TimeDec and SAGE having the same performance shows that decompression is no longer the slowest stage. Third, due to its efficient access patterns, SAGE’s implementation in software (SAGEsw) also leads to speedups ($2.3\times$ on average) over (N)Spr. However, SAGEsw’s decompression still bottlenecks end-to-end performance. SAGEsw leads to up to $4.0\times$ slowdown over SAGE. The hardware implementation of SAGE decompression provides greater speedup than its software version by handling bitwise operations (on arrays and guide arrays) more efficiently and managing data flow more effectively in a fine-grained manner. Ultimately, choosing between these two configurations (software or hardware) is a design decision. SAGE software facilitates its ease of adoption in the near term, while SAGE hardware 1) provides *better performance*, 2) has *significantly better energy reduction* (across *all* our datasets, as shown in Fig. 16), and 3) is very *lightweight*, which makes it suitable

for seamless integration with genome analysis hardware accelerators, even in resource-constrained environments. Fourth, by efficiently integrating SAGE_{SSD} with ISF (i.e., the in-storage filter implemented inside the resource-constrained environment of the SSD), SAGE_{SSD}+ISF leads to $7.8\times$ ($2.5\times$) average speedups over (N)SprAC on the system with the PCIe (SATA) SSD. SAGE_{SSD}+ISF outperforms SAGE in all cases, except when (i) the input and application do not largely take advantage of ISF (i.e., in this case, ISF does not filter many reads in the read set), and (ii) the SSD’s limited external bandwidth bottlenecks performance (e.g., RS1 and RS4 with the SATA SSD). In these cases, the SAGE configuration should be used to decompress data outside the SSD to avoid moving larger decompressed data through the limited-bandwidth storage interface. Fifth, regardless of how much decompression tools are optimized for performance, if their high resource requirements make them unsuitable for adoption in resource-constrained environments, they miss out on the benefits of a wide range of genome analysis systems that are implemented in such constrained environments. For example, 0TimeDec (i.e., an idealized decompressor with zero decompression time, but inefficient for integration in resource-constrained environments) cannot efficiently and cost-effectively integrate with ISF implemented inside the SSD and, as shown in our evaluations, it ends up being on average $1.8\times$ (up to $9.9\times$) slower than SAGE_{SSD}+ISF.

Data Preparation. Fig. 14 shows *only* data preparation throughput, normalized to pigz (on a system with the PCIe SSD). SAGE leads to $91.3\times$, $29.5\times$, and $22.3\times$ average speedups over pigz, (N)Spr, and (N)SprAC, respectively.

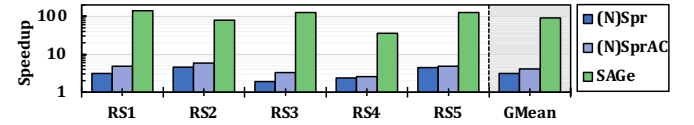


Figure 14: Data preparation speedup.

Multiple SSDs. Since all data in SAGE are accessed in streams, they can be disjointly partitioned between multiple SSDs to enable concurrent access. Fig. 15 shows the speedups of SAGE and SAGE_{SSD}+ISF over (N)Spr in a system with multiple PCIe SSDs. First, SAGE maintains its large speedup over (N)Spr. Second, with more SSDs, SAGE_{SSD}+ISF’s benefits increase for some datasets (RS3 and RS5) since more SSDs improve ISF’s performance, which was on the critical path of the end-to-end execution. We conclude that SAGE is able to effectively leverage multiple SSDs. Given this, and since each genomic read can be mapped independently, SAGE’s design is also compatible for adoption in distributed storage systems.

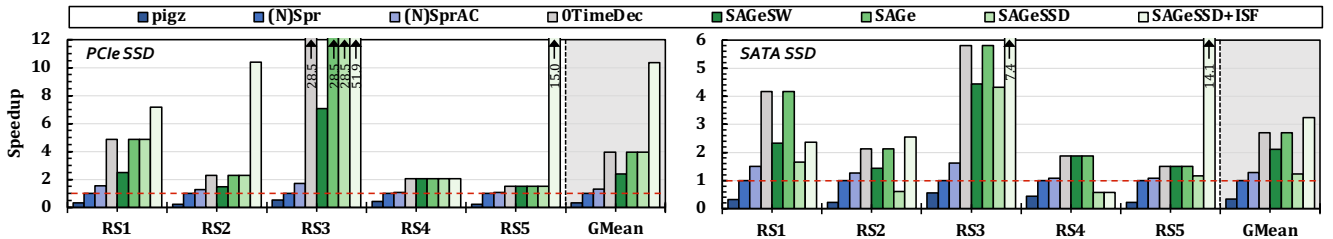


Figure 13: End-to-end speedup for different read sets.

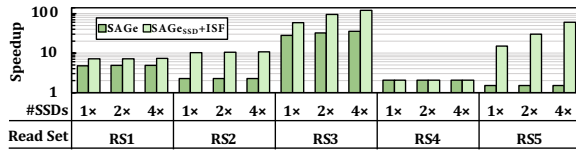


Figure 15: End-to-end speedup with different SSD counts.

8.2. Area and Power

Table 1 lists the area and power of SAGE’s accelerator units at 1 GHz. Although they can operate at a higher frequency, their throughput is already sufficient because SAGE’s accelerator operations are bottlenecked by the NAND flash read throughput. SAGE’s accelerators consume a small area and power of 0.002 mm² and 0.49 mW at 22 nm node.

Table 1: Area and power consumption of SAGE’s logic.

Logic unit	# of instances	Area [mm ²]	Power [mW]
Scan Unit	1 per channel	0.000045	0.014
Read Construction Unit	1 per channel	0.000017	0.023
Double Registers (for mode ③ in Fig. 12)	1 per channel	0.00020	0.035
Control Unit	1 per channel	0.000029	0.025
Total for an 8-channel SSD	-	0.002	0.49 (+0.28 for mode ③)

8.3. Energy

Fig. 16 shows the end-to-end energy reduction, where execution includes *both* data preparation and genome analysis. Energy reduction is normalized to (N)SprAC (higher is better). We observe that SAGE leads to a significant average energy reduction of 34.0×, 16.9×, and 13.0×, over pigz, (N)Spr, and (N)SprAC, respectively.

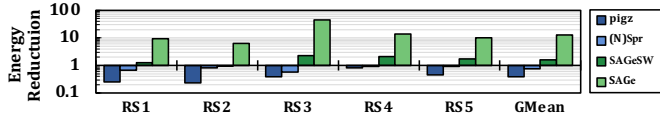


Figure 16: End-to-end energy reduction.

8.4. Compression Ratio

Table 2 shows the compression ratios of pigz, (N)Spr, and SAGE. For DNA bases, SAGE achieves (i) 2.9× better average compression ratio than pigz, and (ii) comparable compression ratios to (N)Spr, with a modest 4.6% average reduction.

Table 2: Compression ratios for different read sets.

Label	Read Set (Short, Long)	Uncomp. Size (MB) DNA+Qual	Compression Ratio			
			PigZ [167]		(Nano)Spring [48]	
			DNA	Qual.	DNA	Qual.
RS1	RRR870667_2 [342] (S)	10 000	3.39	2.23	24.8	2.80
RS2	ERR194146_1 [343] (S)	158 000	12.5	2.49	40.2	3.4
RS3	SR2052419_1 [344] (S)	8 000	3.41	3.45	7.2	5.07
RS4	PAO89685_sampled [345] (L)	24 000	3.93	1.79	4.8	2.19
RS5	ERR545028 [346] (L)	176 800	3.5	1.57	7.6	1.82

Effect of Input Data Characteristics. SAGE can support datasets across different species because the consensus sequence is an approximation of the input sample’s genomes (§2.2). Given datasets from the same species (e.g., RS2 and RS3 in our datasets, which are both *Homo sapiens*), the larger one (R2) achieves higher compression ratios. Since the storage overhead for the consensus sequence is constant, this one-time overhead is more effectively amortized as the number of compressed reads increases.

Effect of Different SAGE Optimizations. To show the impact of different optimizations, Fig. 17¹² presents the size breakdown of the reads’ mismatch information for a short (RS2) and long read set (RS4) in five settings: (i) N0, with no optimization on the raw mismatch information; (ii) 01, with matching position optimization (§5.1.3) added to N0; (iii) 02, with mismatch positions and count optimizations (§5.1.1) added to 01; (iv) 03, with mismatching base and type optimizations (§5.1.2) added to 02; and (v) 04, with corner case optimizations (§5.1.4) added to 03. We make six observations. First, 01 significantly reduces the data size of the matching positions in short reads. Note that 01 is not critical for long reads, since matching positions do not constitute a large fraction of mismatch information in longer reads. Second, 02 significantly reduces the data size of mismatch counts in short reads. This is because, as detailed in §5.1.1, most reads in a short read dataset have no mismatches, benefiting from SAGE’s mismatch count encoding. Third, 02 leads to a large reduction in the mismatch positions’ data size in long reads. Fourth, 03 significantly reduces the bases for long reads by efficiently encoding chimeric reads (§5.1.2). However, the data size for mismatch positions increases in 03 due to the additional mismatches at the chimeric reads’ new matching positions. This tradeoff is suitable for SAGE, as it can efficiently encode the greater number of mismatch positions. Fifth, 03 reduces the types’ data size for short and long reads. Sixth, 04 reduces the data size required for labeling corner cases.

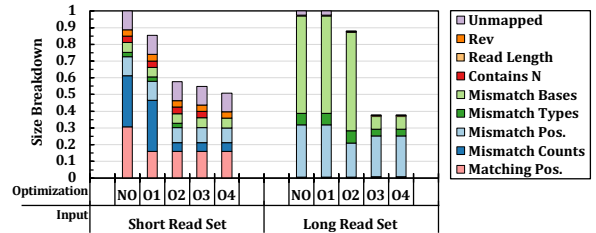


Figure 17: Effect of different SAGE optimizations on the storage size of mismatch information.

Quality Scores. SAGE achieves (i) 33% better average compression ratio than pigz, and (ii) the same ratios as (N)Spr. This is because, as mentioned in §5.1.5, SAGE’s quality score (de)compression is based on the same software (de)compression used for quality scores in (N)Spr.

8.5. Resource Requirements

Table 3 quantifies SAGE’s lightweightness and effectiveness by demonstrating the resource requirements of SAGE and other tools. This table lists the state-of-the-art general-purpose and genomics-specific compression tools implemented on GPUs, FPGAs, ASICs, and CPUs.¹³ We make three observations. First, leveraging its synergistic, data-aware co-design, SAGE yields the largest average decompression throughput. Second, SAGE achieves a significantly higher compression ratio than general-purpose decompressors and a comparable one to genomics-specific decompressors (and higher than the GPU-based genomics-specific decompressor [53]). Third, SAGE eliminates

¹²Unmapped refers to reads that do not match to the consensus sequence, and Rev refers to a bit marking if a read matches in reverse to the consensus.

¹³We report compression ratio based on the ratio achieved by each tool’s compression algorithm on our datasets. For decompression throughput, we report the best performance reported by the original works.

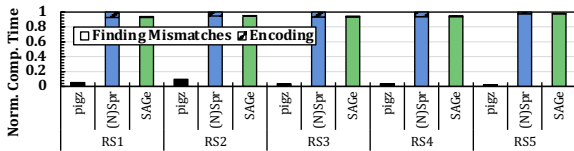
Table 3: Comparison of decompression tools.

Tool	Genomics specific	Avg. Comp. Ratio	End-to-end?	Hardware Requirements	Memory Footprint	Decomp. Throughput (GB/s)
nvCOMP (DEFLATE) [347]	×	5.3	✓	GPU (NVIDIA A100 [348])	1.5 GB	50
Xilinx/AMD GZIP Engine [349]	×	5.3	✓	FPGA (AMD Alveo U50 [350])	80 KB	0.7
xz [274]	×	6.7	✓	CPU (AMD® EPYC® 7742 CPU [324] with 128 physical cores)	13 GB	0.6
HW-accelerated ZS-standard [351]	×	6.7	✓	ASIC (1.89 mm ² in a 14nm technology node)	2–64 KB	3.9
GPUFastqLZ [53]	✓	5.8	✓	GPU (A GPU server with four NVIDIA Tesla V100 GPUs [352])	N/A ¹⁴	7.8
repaq [54]	✓	17.1	×	FPGA (AMD ALVEO U200 [353])	16 GB	N/A
Spring [43] & NanoSpring [48]	✓	16.9	✓	CPU (AMD® EPYC® 7742 CPU [324] with 128 physical cores)	26 GB	0.7
SAGE	✓	15.8	✓	ASIC (0.002 mm ² in a 22nm technology node)	128 B	75.4

the need for costly and power-hungry computational resources and large memory capacity or bandwidth since, unlike many decompression techniques, it avoids frequent random accesses for matching patterns in large data structures.

8.6. Compression Time

Compression of genomic data is typically an offline and one-time task and is *not* in the critical path of genome analysis. For completeness, Fig. 18 reports the compression times of pigz, (N)Spr, and SAGE. In all cases, compression is performed on the host CPU. We make two observations. First, SAGE’s compression is slightly faster than (N)Spr. Compression times of both SAGE and (N)Spr are mainly dominated by finding mismatch information. After this step, (N)Spr uses back-end general-purpose compression techniques to compress the mismatch information, while SAGE uses the procedure discussed in §5.1 to compress the mismatch information. Second, genomics compressors (both (N)Spr and SAGE) have much longer compression times than general-purpose compressors due to the need to find mismatches. As mentioned in §2.2, mismatch information enables genomics compressors to achieve substantially better compression ratios by finding long-range similarity patterns that cannot be captured by general compressors.


Figure 18: Normalized compression time.

9. Related Work

To our knowledge, SAGE is the *first* system to mitigate the data preparation bottleneck in genome sequence analysis. SAGE achieves compression ratios comparable to state-of-the-art genomic compressors and maintaining a lightweight design for integration with a wide range of genome analysis systems. **Accelerating genome sequence analysis.** Many prior works accelerate genome sequence analysis via algorithmic optimizations (e.g., [79–85]), or hardware acceleration (e.g. [76–80,86–140,142–148,159,183,238–255]). SAGE is orthogonal to these works and can be flexibly integrated with them to further improve their benefits.

Genomics-Specific Compression. Many works (e.g., [43–59,276,354]) propose genomic compression. As detailed in §3.2, some works (e.g., [54,151–158]) accelerate certain computational kernels widely used in genomic compressors. Despite their benefits, these approaches (i) are unsuitable for resource-constrained environments and/or (ii) do *not* fully mitigate the end-to-end preparation bottleneck (evaluated in §8). While some works [281,282] use algorithms that do not rely on expensive resources, they compress poorly (§3.2). As a strongly conservative evaluation, Θ TimeDec (in Fig. 13) can serve as an idealized representation of decompressors that, despite their optimized performance, cannot be integrated in resource-constrained environments. As shown in §8.1, by integration in a resource-constrained environment, SAGE_{SSD}+Ideal-ISF leads to significant performance benefits over Θ TimeDec.

General-Purpose Compression. Many works propose general-purpose compression in software (e.g., [60–66]) or hardware (e.g., [67–75,355–362]). However, as analyzed in §2.2, general-purpose compressors achieve poor compression ratios for genomic data [39,168].

10. Conclusion

We propose SAGE, an algorithm-system co-design for highly-compressed storage and high-performance access of large-scale genomic data, to mitigate the data preparation bottleneck in genome analysis. We leverage properties of genomic data to co-design SAGE’s algorithm and architecture, such that highly-compressed data can be efficiently interpreted by lightweight hardware and rapidly prepared for analysis. Due to its lightweight design, SAGE can be seamlessly integrated with a broad range of genome analysis systems to mitigate their data preparation bottlenecks and unlock the full potential of genome analysis acceleration. Our evaluations show that SAGE significantly improves the end-to-end performance and energy efficiency of state-of-the-art genome sequence analysis accelerators, compared to when the accelerators rely on state-of-the-art software and hardware decompression tools. We conclude that addressing the data preparation bottleneck is crucial for unlocking the full potential of genome analysis acceleration, and hope that SAGE will inspire new studies and designs in this relatively unexplored research area.

Acknowledgments

We thank the anonymous reviewers of HPCA 2025, ISCA 2025, MICRO 2025, and HPCA 2026 for feedback. We thank the SAFARI group members for feedback and the stimulating, inclusive, intellectual, and scientific environment. We acknowledge the generous gifts and support provided by our industrial partners, including Google, Huawei, Intel, Microsoft, and VMware. This research was partially supported by European Union’s Horizon Programme for research and innovation under Grant Agreement No. 101047160 (project BioPIM), the Swiss National Science Foundation (SNSF), Semiconductor Research Corporation (SRC), the ETH Future Computing Laboratory (EFCL), Huawei ZRC Storage Team, and the AI Chip Center for Emerging Smart Systems Limited (ACCESS). Jisung Park was supported by the NRF (RS-2025-00519994) and the IITP (RS-2024-00459026) of Korea.

References

- [1] Michelle M. Clark, Amber Hildreth, Sergey Batalov, Yan Ding, Shimul Chowdhury, Kelly Watkins, Katarzyna Ellsworth, Brandon Camp, Cyrielle I. Kint, Calum Ya-coubian, Lauge Farnaes, Matthew N. Bainbridge, Curtis Beebe, Joshua J. A. Braun, Margaret Bray, Jeanne Carroll, Julie A. Cakici, Sara A. Caylor, Christina Clarke, Mitchell P. Creed, Jennifer Friedman, Alison Frith, Richard Gain, Mary Gaughran, Shauna George, Sheldon Gilmer, Joseph Gleeson, Jeremy Gore, Haiying Grunenwald, Raymond L. Hovey, Marie L. Janes, Kejia Lin, Paul D. McDonagh, Kyle McBride, Patrick Mulrooney, Shareef Nahas, Daeheon Oh, Albert Oriol, Laura Puckett, Zia Rady, Martin G. Reese, Julie Ryu, Lisa Salz, Erica Sanford, Lawrence Stewart, Nathaly Sweeney, Mari Tokita, Luca Van Der Kraan, Sarah White, Kristen Wigby, Brett Williams, Terence Wong, Meredith S. Wright, Catherine Yamada, Peter Schols, John Reynders, Kevin Hall, David Dimmock, Narayanan Veeraraghavan, Thomas Defay, and Stephen F. Kingsmore. Diagnosis of Genetic Diseases in Seriously Ill Children by Rapid Whole-genome Sequencing and Automated Phenotyping and Interpretation. *Science Translational Medicine*, 2019.
- [2] Lauge Farnaes, Amber Hildreth, Nathaly M. Sweeney, Michelle M. Clark, Shimul Chowdhury, Shareef Nahas, Julie A. Cakici, Wendy Benson, Robert H. Kaplan, Richard Kronick, Matthew N. Bainbridge, Jennifer Friedman, Jeffrey J. Gold, Yan Ding, Narayanan Veeraraghavan, David Dimmock, and Stephen F. Kingsmore. Rapid Whole-genome Sequencing Decreases Infant Morbidity and Cost of Hospitalization. *NPJ Genomic Medicine*, 2018.
- [3] Nathaly M. Sweeney, Shareef A. Nahas, Shimul Chowdhury, Sergey Batalov, Michelle Clark, Sara Caylor, Julie Cakici, John J. Nigro, Yan Ding, Narayanan Veeraraghavan, Charlotte Hobbs, David Dimmock, and Stephen F. Kingsmore. Rapid Whole Genome Sequencing Impacts Care and Resource Utilization in Infants with Congenital Heart Disease. *NPJ Genomic Medicine*, 2021.
- [4] Can Alkan, Jeffrey M Kidd, Tomas Marques-Bonet, Gozde Aksay, Francesca Antonacci, Fereydoun Hormozdiani, Jacob O Kitzman, Carl Baker, Maika Malig, Onur Mutlu, S Cenk Sahinalp, Richard A Gibbs, and Evan E Eichler. Personalized Copy Number and Segmental Duplication Maps Using Next-Generation Sequencing. *Nature Genetics*, 2009.
- [5] Mauricio Flores, Gustavo Glusman, Kristin Brogaard, Nathan D Price, and Leroy Hood. P4 Medicine: How Systems Medicine Will Transform the Healthcare Sector and Society. *Personalized Medicine*, 2013.
- [6] Geoffrey S Ginsburg and Huntington F Willard. Genomic and Personalized Medicine: Foundations and Applications. *Translational Research*, 2009.
- [7] Lynda Chin, Jannik N Andersen, and P Andrew Futreal. Cancer Genomics: From Discovery Science to Personalized Medicine. *Nature Medicine*, 2011.
- [8] Euan A Ashley. Towards Precision Medicine. *Nature Reviews Genetics*, 2016.
- [9] Joshua S. Bloom, Laila Sathe, Chetan Munugala, Eric M. Jones, Molly Gasperini, Nathan B. Lubock, Fauna Yarza, Erin M. Thompson, Kyle M. Kovary, Jimin Park, Dawn Marquette, Stephanie Kay, Mark Lucas, TreQuan Love, A. Sina Boeshaghgi, Oliver F. Brandenburg, Longhua Guo, James Boockock, Myles Hochman, Scott W. Simpkins, Isabella Lin, Nathan LaPierre, Duke Hong, Yi Zhang, Gabriel Oland, Bianca Judy Choe, Sukantha Chandrasekaran, Evann E. Hilt, Manish J. Butte, Robert Damoiseaux, Clifford Kravitz, Aaron R. Cooper, Yi Yin, Lior Pachter, Omai B. Garner, Jonathan Flint, Eleazar Eskin, Chongyuan Luo, Sriram Kosuri, Leonid Kruglyak, and Valerie A. Arboleda. Massively Scaled-up Testing for SARS-CoV-2 RNA via Next-generation Sequencing of Pooled and Barcoded Nasal and Saliva Samples. *Nature Biomedical Engineering*, 2021.
- [10] Ramesh Yelagandula, Aleksandr Bykov, Alexander Vogt, Robert Heinen, Ezgi Özkan, Marcus Martin Strobl, Juliane Christina Baar, Kristina Uzunova, Bence Hajdusits, Darja Kordic, Erna Suljic, Amina Kurtovic-Kozaric, Sebija Izetbegovic, Justine Schaeffer, Peter Hufnagl, Alexander Zoufaly, Tamara Seitz, Mariam Al-Rawi, Stefan Ameres, Juliane Baar, Benedikt Bauer, Nikolaus Beer, Katharina Bergauer, Wolfgang Binder, Claudia Blaukopf, Boril Bochev, Julius Brennecke, Selina Brinnich, Aleksandra Bundalo, Meinrad Busslinger, Tim Clausen, Geert de Vries, Marcus Dekens, David Drechsel, Zuzana Dzipinkova, Michaela Eckmann-Mader, Michaela Fellner, Thomas Fellner, Laura Fin, Michaela Valeria Gapp, Gerlinde Grabmann, Irina Grishkovskaya, Astrid Hagelkruys, Dominik Handler, David Haselbach, Louisa Hempel, Louisa Hill, David Hoffmann, Stefanie Horer, Harald Isemann, Robert Kalis, Max Kellner, Juliane Kley, Thomas Köcher, Alwin Köhler, Christian Krauditsch, Sabina Kula, Sonja Lang, Richard Latham, Marie-Christin Leitner, Thomas Leonard, Dominik Lindenhof, Raphael Arthur Manzenreither, Martin Matl, Karl Mechtler, Anton Meinhardt, Stefan Mereiter, Thomas Micheler, Paul Moeseneder, Tobias Neumann, Simon Nimpf, Magnus Nordborg, Egon Ogris, Michaela Pagani, Andrea Pauli, Jan-Michael Peters, Petra Pjevack, Clemens Plaschka, Martina Rath, Daniel Reumann, Sarah Rieser, Marianne Rocha-Hasler, Alan Rodriguez, Nathalie Ropek, James Julian Ross, Harald Scheuch, Karina Schindler, Clara Schmidt, Hannes Schmidt, Jakob Schnabl, Stefan Schüchner, Tanja Schwickert, Andreas Sommer, Daniele Soldoroni, Johannes Stadlmann, Peter Steinlein, Marcus Strobl, Simon Strobl, Qiong Sun, Wen Tang, Linda Trübestein, Johanna Trupke, Christian Umkehrer, Sandor Urmosi-Ince, Gijs Versteeg, Vivien Vogt, Michael Wagner, Martina Weissenboeck, Barbara Werner, Johannes Zuber, Manuela Födinger, Franz Allerberger, Alexander Stark, Luisa Cochella, Ulrich Elling, and VCDI. Multiplexed Detection of SARS-CoV-2 and Other Respiratory Infections in High Throughput by SARSeq. *Nature Communications*, 2021.
- [11] Vien Thi Minh Le and Binh An Diep. Selected Insights from Application of Whole Genome Sequencing for Outbreak Investigations. *Current Opinion in Critical Care*, 2013.
- [12] Vlad Nikolayevskiy, Katharina Kranzer, Stefan Niemann, and Francis Drobniowski. Whole Genome Sequencing of Mycobacterium Tuberculosis for Detection of Recent Transmission and Tracing Outbreaks: A Systematic Review. *Tuberculosis*, 2016.
- [13] Shaofu Qiu, Peng Li, Hongbo Liu, Yong Wang, Nan Liu, Chengyi Li, Shenlong Li, Ming Li, Zhengjie Jiang, Huandong Sun, Ying Li, Jing Xie, Chaojie Yang, Jian Wang, Hao Li, Shengjie Yi, Zhihao Wu, Leili Jia, Ligui Wang, Rongzhang Hao, Yansong Sun, Liuyu Huang, Hui Ma, Zhengquan Yuan, and Hongbin Song. Whole-genome Sequencing for Tracing the Transmission Link between Two ARD Outbreaks Caused by A Novel HAdV Serotype 7 Variant, China. *Scientific Reports*, 2015.
- [14] Carol A Gilchrist, Stephen D Turner, Margaret F Riley, William A Petri, and Erik L Hewlett. Whole-genome Sequencing in Outbreak Analysis. *Clinical Microbiology Reviews*, 2015.
- [15] NIHR Global Health Research Unit on Genomic Surveillance of AMR. Whole-genome sequencing as part of national and international surveillance programmes for antimicrobial resistance: a roadmap. *BMJ Global Health*, 2020.
- [16] Shanwei Tong, Luyao Ma, Jennifer Ronholm, William Hsiao, and Xiaonan Lu. Whole genome sequencing of Campylobacter in agri-food surveillance. *Current Opinion in Food Science*, 2021.
- [17] Shiv Prasad, Lal Chand Malav, Jairam Choudhary, Sudha Kannojiya, Monika Kundu, Sandeep Kumar, and Ajar Nath Yadav. Soil Microbiomes for Healthy Nutrient Recycling. *Current Trends in Microbial Biotechnology for Sustainable Agriculture*, 2021.
- [18] Martin Mascher, Murukarthick Jayakodi, Hyeonah Shim, and Nils Stein. Promises and challenges of crop translational genomics. *Nature*, 2024.
- [19] Mona Schreiber, Murukarthick Jayakodi, Nils Stein, and Martin Mascher. Plant pangomes for crop improvement, biodiversity and evolution. *Nature Reviews Genetics*, 2024.
- [20] Aneta K. Urbanek, Waldemar Rymowicz, and Aleksandra M. Mironczuk. Degradation of plastics and plastic-degrading bacteria in cold marine habitats. *Applied Microbiology and Biotechnology*, 2018.
- [21] Robert C. Edgar, Jeff Taylor, Victor Lin, Tomer Altman, Pierre Barbera, Dmitry Meleshko, Dan Lohr, Gherman Novakovsky, Benjamin Buchfink, Basem Al-Shayeb, Jillian F. Banfield, Marcos de la Peña, Anton Korobeynikov, Rayan Chikhi, and Artem Babaian. Petabase-scale sequence alignment catalyses viral discovery. *Nature*, 2022.
- [22] Lucas Paoli, Hans-Joachim Ruscheweyh, Clarissa C. Forneris, Florian Hubrich, Satria Kautsar, Agneya Bhushan, Alessandro Lotti, Quentin Claysen, Guillem Salazar, Alessio Milanese, Charlotte I. Carlström, Chrysa Papadopoulou, Daniel Gehrig, Mikhail Karasikov, Harun Mustafa, Martin Larralde, Laura M. Carroll, Pablo Sánchez, Ahmed A. Zayed, Dylan R. Cronin, Silvia G. Acinas, Peer Bork, Chris Bowler, Tom O. Delmont, Josep M. Gasol, Alvar D. Gossert, André Kahles, Matthew B. Sullivan, Patrick Wincker, Georg Zeller, Serina L. Robinson, Jörn Piel, and Shinichi Sunagawa. Biosynthetic potential of the global ocean microbiome. *Nature*, 2022.
- [23] Carolyn J. Hogg. Translating genomic advances into biodiversity conservation. *Nature Reviews Genetics*, 2024.
- [24] Harris A. Lewin, Gene E. Robinson, W. John Kress, William J. Baker, Jonathan Coddington, Keith A. Crandall, Richard Durbin, Scott V. Edwards, Félix Forest, M. Thomas P. Gilbert, Melissa M. Goldstein, Igor V. Grigoriev, Kevin J. Hackett, David Haussler, Erich D. Jarvis, Warren E. Johnson, Aristides Patrinos, Stephen Richards, Juan Carlos Castilla-Rubio, Marie-Anne van Sluys, Pamela S. Soltis, Xun Xu, Huanming Yang, and Guojie Zhang. Earth BioGenome Project: Sequencing life for the future of life. *Proceedings of the National Academy of Sciences*, 2018.
- [25] Hans Ellegren. Genome Sequencing and Population Genomics in Non-Model Organisms. *Trends in Ecology & Evolution*, 2014.
- [26] Javier Prado-Martinez, Peter H. Sudmant, Jeffrey M. Kidd, Heng Li, Joanna L. Kelley, Belen Lorente-Galdos, Krishna R. Veeramah, August E. Woerner, Timothy D. O'Connor, Gabriel Sanpere, Alexander Cagan, Christoph Theunert, Ferran Casals, Hafid Laayouni, Kasper Munch, Asger Holobolth, Anders E. Halager, Maika Malig, Jessica Hernandez-Rodriguez, Irene Hernandez-Herraez, Kay Prüfer, Marc Pybus, Laurel Johnstone, Michael Lachmann, Can Alkan, Dorina Twigg, Natalia Petit, Carl Baker, Fereydoun Hormozdiani, Marcos Fernandez-Callejo, Marc Dabad, Michael L. Wilson, Laurie Stevenson, Cristina Camprubi, Tiago Carvalho, Aurora Ruiz-Herrera, Laura Vives, Marta Mele, Teresa Abello, Ivanela Kondova, Ronald E. Bontrop, Anne Pusey, Felix Lankester, John A. Kiyang, Richard A. Bergl, Elizabeth Lonsdorf, Simon Myers, Mario Ventura, Pascal Gagneux, David Comas, Hans Siegmund, Julie Blanc, Lidia Agueda-Calpena, Marta Gut, Lucinda Fulton, Sarah A. Tishkoff, James C. Mullikin, Richard K. Wilson, Ivo G. Gut, Mary Katherine Gonder, Oliver A. Ryder, Beatrice H. Hahn, Arcadi Navarro, Joshua M. Akey, Jaume Bertranpetit, David Reich, Thomas Mailund, Mikkel H. Schierup, Christina Hvilsom, Aida M. Andrés, Jeffrey D. Wall, Carlos D. Bustamante, Michael F. Hammer, Evan E. Eichler, and Tomas Marques-Bonet. Great Ape Genetic Diversity and Population History. *Nature*, 2013.
- [27] Ana Prohaska, Fernando Racimo, Andrew J Schork, Martin Sikora, Aaron J Stern, Melissa Ilardo, Morten Erik Allentoft, Lasse Folkersen, Alfonso Buil, J Victor Moreno-Mayar, Thorfinn Korneliussen, Daniel Geschwind, Andrés Ingason, Thomas Werge, Rasmus Nielsen, and Eske Willerslev. Human Disease Variation in the Light of Population Genomics. *Cell*, 2019.
- [28] David Danko, Daniela Bezdán, Evan E. Afshin, Sofia Ahsanuddin, Chandrima Bhattacharya, Daniel J. Butler, Kern Rei Chng, Daisy Donnellan, Jochen Hecht, Katelyn Jackson, Katerina Kuchin, Mikhail Karasikov, Abigail Lyons, Lauren Mak, Dmitry Meleshko, Harun Mustafa, Beth Mutai, Russell Y. Neches, Amanda Ng, Olga Nikolayeva, Tatyana Nikolayeva, Eileen Png, Krista A. Ryon, Jorge L. Sanchez, Heba Shaaban, Maria A. Sierra, Dominique Thomas, Ben Young, Omar O. Abudayyeh, Josue Alica, Malay Bhattacharyya, Ran Blekhan, Eduardo Castro-Nallar, Ana M. Cañas, Aspasia D. Chatziefthimiou, Robert W. Crawford, Francesca De Filippis, Youping Deng, Christelle Desnues, Emmanuel Dias-Neto, Marius Dybwad, Eran Elhaik, Danilo Ercolini, Alina Frolova, Dennis Gankin, Jonathan S. Gootenberg, Alexandra B. Graf, David C. Green, Iman Hajirasouliha, Jaden J.A. Hastings, Mark

- Hernandez, Gregorio Iraola, Soojin Jang, Andre Kahles, Frank J. Kelly, Kaymisha Knights, Nikos C. Kyrpides, Patrick P. Labaj, Patrick K.H. Lee, Marcus H.Y. Leung, Per O. Ljungdahl, Gabriella Mason-Buck, Ken McGrath, Cem Meydan, Emmanuel F. Mongodin, Milton Ozorio Moraes, Niranjana Nagarajan, Marina Nieto-Caballero, Houtan Noushmehr, Manuela Oliveira, Stephen Ossowski, Olayinka O. Osuolale, Orhan Özcan, David Paez-Espino, Nicolás Rascovan, Hugues Richard, Gunnar Rätsch, Lynn M. Schriml, Torsten Semmler, Osman U. Sezerman, Leming Shi, Tielu Shi, Rania Siam, Le Huu Song, Haruo Suzuki, Denise Syndercombe Court, Scott W. Tighe, Xinzhaoh Tong, Klas I. Udekwu, Juan A. Ugalde, Brandon Valentine, Dimitar I. Vassilev, Elena M. Vayndorf, Thirumalaisamy P. Velavan, Jun Wu, Maria M. Zambano, Jifeng Zhu, Sibo Zhu, Christopher E. Mason, and International MetaSUB Consortium. A Global Metagenomic Map of Urban Microbiomes and Antimicrobial Resistance. *Cell*, 2021.
- [29] Xavier Didelot, Rory Bowden, Daniel J Wilson, Tim EA Peto, and Derrick W Crook. Transforming clinical microbiology with bacterial genome sequencing. *Nature Reviews Genetics*, 2012.
- [30] Simone Marini, Rodrigo A Mora, Christina Boucher, Noelle Robertson Noyes, and Mattia Prosperi. Towards routine employment of computational tools for antimicrobial resistance determination via high-throughput sequencing. *Briefings in Bioinformatics*, 2022.
- [31] Charlotte J. Houldcroft, Mathew A. Beale, and Judith Breuer. Clinical and biological insights from viral genome sequencing. *Nature Reviews Microbiology*, 2017.
- [32] Natasha Jansz and Geoffrey J. Faulkner. Viral genome sequencing methods: benefits and pitfalls of current approaches. *Biochemical Society Transactions*, 2024.
- [33] Bonnie Berger and Yun William Yu. Navigating bottlenecks and trade-offs in genomic data analysis. *Nature Reviews Genetics*, 2023.
- [34] Kenneth Katz, Oleg Shutov, Richard Lapoint, Michael Kimelman, J Rodney Brister, and Christopher O'Sullivan. The Sequence Read Archive: a decade more of explosive growth. *Nucleic Acids Research*, 2021.
- [35] Josephine Burgin, Alisha Ahmed, Carla Cummins, Rajkumar Devraj, Khadim Gueye, Dipayan Gupta, Vikas Gupta, Muhammad Haseeb, Maira Ihsan, Eugene Ivanov, Suran Jayatilaka, Vishnukumar Balavenkataraman Kadhirvelu, Manish Kumar, Ankur Lathi, Rasko Leinonen, Milena Mansurova, Jasmine McKinnon, Colman O'Cathail, Joana Paupério, Stéphane Pesant, Nadim Rahman, Gabriele Rinck, Sandeep Selvakumar, Swati Suman, Senthilnathan Vijayaraja, Zahra Waheed, Peter Woollard, David Yuan, Ahmad Zyou, Tony Burdett, and Guy Cochrane. The European Nucleotide Archive in 2022. *Nucleic Acids Research*, 2022.
- [36] National Center for Biotechnology Information. SRA database growth. <https://www.ncbi.nlm.nih.gov/sra/docs/sragrowth/>, 2024.
- [37] European Bioinformatics Institute. European Nucleotide Archive Statistics. <https://www.ebi.ac.uk/ena/browser/about/statistics>, 2023.
- [38] Zachary D. Stephens, Skylar Y. Lee, Faraz Faghri, Roy H. Campbell, Chengxiang Zhai, Miles J. Efron, Ravishanker Iyer, Michael C. Schatz, Saurabh Sinha, and Gene E. Robinson. Big Data: Astronomical or Genomic? *PLOS Biology*, 2015.
- [39] Zexuan Zhu, Yongpeng Zhang, Zhen Ji, Shan He, and Xiao Yang. High-throughput DNA sequence data compression. *Briefings in Bioinformatics*, 2013.
- [40] Sebastian Deorowicz and Szymon Grabowski. Data compression for sequencing data. *Algorithms for Molecular Biology*, 2013.
- [41] Raffaele Giancarlo, Simona E. Rombo, and Filippo Utro. Compressive biological sequence analysis and archival in the era of high-throughput sequencing technologies. *Briefings in Bioinformatics*, 2013.
- [42] Raphael O. Betschart, Felix Thalén, Stefan Blankenberg, Martin Zoche, Tanja Zeller, and Andreas Ziegler. A benchmark study of compression software for human short-read sequence data. *Scientific Reports*, 2025.
- [43] Shubham Chandak, Kedar Tatwawadi, Idoia Ochoa, Mikel Hernaez, and Tsachy Weissman. SPRING: a next-generation compressor for FASTQ data. *Bioinformatics*, 2018.
- [44] Shubham Chandak, Kedar Tatwawadi, and Tsachy Weissman. Compression of genomic sequencing reads via hash-based reordering: algorithm and analysis. *Bioinformatics*, 2017.
- [45] Łukasz Roguski, Idoia Ochoa, Mikel Hernaez, and Sebastian Deorowicz. FaStore: a space-saving solution for raw sequencing data. *Bioinformatics*, 2018.
- [46] Vinicius Cogo, João Paulo, and Alysso Bessani. GenoDedup: Similarity-Based Deduplication and Delta-Encoding for Genome Sequencing Data. *IEEE TC*, 2021.
- [47] Marek Kokot, Adam Gudyś, Heng Li, and Sebastian Deorowicz. CoLoRd: compressing long reads. *Nature Methods*, 2022.
- [48] Qingxi Meng, Shubham Chandak, Yifan Zhu, and Tsachy Weissman. Reference-free lossless compression of nanopore sequencing reads using an approximate assembly approach. *Scientific Reports*, 2023.
- [49] Dufort y Álvarez, Guillermo and Seroussi, Gadiel and Smircich, Pablo and Sotelo-Silveira, José and Ochoa, Idoia and Martín, Álvaro. RENANO: a REFERENCE-based compressor for NANOpore FASTQ files. *Bioinformatics*, 2021.
- [50] Tomasz M Kowalski and Szymon Grabowski. pGRc: pseudogenome-based read compressor. *Bioinformatics*, 2019.
- [51] Guillermo Dufort y Álvarez, Gadiel Seroussi, Pablo Smircich, José Sotelo, Idoia Ochoa, and Álvaro Martín. ENANO: Encoder for NANOpore FASTQ files. *Bioinformatics*, 2020.
- [52] Illumina. DRAGEN ORA Compression and Decompression. https://support-docs.illumina.com/SW/DRAGEN_v38/Content/SW/DRAGEN_ORA_Compression_fDG_swHS.htm, 2021.
- [53] Taolue Yang, Youyuan Liu, Bo Jiang, and Sian Jin. GpuFastqLZ: An Ultra Fast Compression Methodology for Fastq Sequence Data on GPUs. In *SC-W*, 2024.
- [54] Shifu Chen, Yaru Chen, Zhouyang Wang, Wenjian Qin, Jing Zhang, Heera Nand, Jishuai Zhang, Jun Li, Xiaoni Zhang, Xiaoming Liang, and Mingyan Xu. Efficient sequencing data compression and FPGA acceleration based on a two-step framework. *Frontiers in Genetics*, 2023.
- [55] Faraz Hach, Ibrahim Numanagić, Can Alkan, and S Cenk Sahinalp. SCALCE: boosting sequence compression algorithms using locally consistent encoding. *Bioinformatics*, 2012.
- [56] Łukasz Roguski and Sebastian Deorowicz. DSRC 2—Industry-oriented compression of FASTQ files. *Bioinformatics*, 2014.
- [57] Sebastian Deorowicz. FQsqueezer: k-mer-based compression of sequencing data. *Scientific Reports*, 2020.
- [58] Divon Lan, Ray Tobler, Yassine Souilmi, and Bastien Llamas. Genozip: a universal extensible genomic data compressor. *Bioinformatics*, 2021.
- [59] Sultan Al Yami and Chun-Hsi Huang. LFASTQC: A lossless non-reference-based FASTQC compressor. *PLOS One*, 2019.
- [60] Y. Collet and M. Kuchera. RFC 8878: Zstandard Compression and the 'application/zstd' Media Type, 2021.
- [61] Igor Pavlov. 7-Zip. <https://7-zip.org/>, 2016.
- [62] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obyrk, Zoltan Szabadka, and Lode Vandevenne. Brotli: A General-Purpose Data Compressor. *ACM TOIS*, 2018.
- [63] Phillip W. Katz. String searcher, and compressor using same. US Patent US5051745A, 1991.
- [64] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. DZIP: Improved general-purpose loss less compression based on novel neural network modeling. In *DCC*, 2021.
- [65] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. DeepZip: Lossless Data Compression Using Recurrent Neural Networks. In *DCC*, 2019.
- [66] Xiang Chen, Tao Lu, Jiapi Wang, Yu Zhong, Guangchun Xie, Xueming Cao, Yuanpeng Ma, Bing Si, Feng Ding, Ying Yang, Yunxin Huang, Yafei Yang, You Zhou, and Fei Wu. HA-CSD: Host and SSD Coordinated Compression for Capacity and Performance. In *IPDPS*, 2024.
- [67] Matěj Bartík, Sven Ubik, and Pavel Kubalik. LZ4 compression algorithm on FPGA. In *ICECS*, 2015.
- [68] Weiqiang Liu, Faqiang Mei, Chenghua Wang, Maire O'Neill, and Earl E Swartzlander. Data compression device based on modified LZ4 algorithm. *IEEE TCE*, 2018.
- [69] Jeremy Fowers, Joo-Young Kim, Doug Burger, and Scott Hauck. A scalable high-bandwidth architecture for lossless compression on FPGAs. In *FCCM*, 2015.
- [70] Jianyu Chen, Maurice Daverveldt, and Zaid Al-Ars. FPGA acceleration of zstd compression algorithm. In *IPDPSW*, 2021.
- [71] Alexandra Angerd, Angelos Arelakis, Vasilis Spiliopoulos, Erik Sintorn, and Per Stenström. GBDI: Going beyond base-delta-immediate compression with global bases. In *HPCA*, 2022.
- [72] Ruihao Gao, Zhichun Li, Guangming Tan, and Xueqi Li. BeeZip: Towards An Organized and Scalable Architecture for Data Compression. In *ASPLOS*, 2024.
- [73] Sagar Karandikar, Aniruddha N. Udiipi, Junsun Choi, Joonho Whangbo, Jerry Zhao, Sviilen Kanev, Edwin Lim, Jyrki Alakuijala, Vrishabh Madduri, Yakun Sophia Shao, Borivoje Nikolic, Krste Asanovic, and Parthasarathy Ranganathan. CDPU: Co-designing compression and decompression processing units for hyperscale systems. In *ISCA*, 2023.
- [74] Yifan Yang, Joel S. Emer, and Daniel Sanchez. SpZip: Architectural Support for Effective Data Compression In Irregular Applications. In *ISCA*, 2021.
- [75] Bulent Abali, Bart Blanner, John Reilly, Matthias Klein, Ashutosh Mishra, Craig B. Agricola, Bedri Sendir, Alper Buyuktosunoglu, Christian Jacobi, William J. Starke, Haren Myneni, and Charlie Wang. Data compression accelerator on IBM POWER9 and z15 processors: Industrial product. In *ISCA*, 2020.
- [76] Mohammed Alser, Zülal Bingöl, Damla Senol Cali, Jeremie Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Accelerating genome analysis: A primer on an ongoing journey. *IEEE Micro*, 2020.
- [77] Mohammed Alser, Joel Lindegger, Can Firtina, Nour Almadhoun, Haiyu Mao, Gagandeep Singh, Juan Gomez-Luna, and Onur Mutlu. From molecules to genomic variations: Accelerating genome analysis via intelligent algorithms and architectures. *Computational and Structural Biotechnology Journal*, 2022.
- [78] Onur Mutlu and Can Firtina. Accelerating Genome Analysis via Algorithm-Architecture Co-Design. In *DAC*, 2023.
- [79] Zheng Zhang, Scott Schwartz, Lukas Wagner, and Webb Miller. A Greedy Algorithm for Aligning DNA Sequences. *Journal of Computational Biology*, 2000.
- [80] Guy St C Slater and Ewan Birney. Automated Generation of Heuristics for Biological Sequence Comparison. *BMC Bioinformatics*, 2005.
- [81] Heng Li. Minimap2: Pairwise Alignment for Nucleotide Sequences. *Bioinformatics*, 2018.
- [82] Gene Myers. A Fast Bit-vector Algorithm for Approximate String Matching Based on Dynamic Programming. *JACM*, 1999.
- [83] Santiago Marco-Sola, Juan Carlos Moure, Miquel Moreto, and Antonio Espinosa. Fast Gap-affine Pairwise Alignment Using the Wavefront Algorithm. *Bioinformatics*, 2021.
- [84] Santiago Marco-Sola, Jordan M Eizenga, Andrea Guarracino, Benedict Paten, Erik Garrison, and Miquel Moreto. Optimal gap-affine alignment in O(s) space. *Bioinformatics*, 2023.
- [85] Ragnar Groot Koerkamp. A*PA2: Up to 19× Faster Exact Global Alignment. In *WABI*, 2024.
- [86] Hongyi Xin, Donghyuk Lee, Farhad Hormozdiari, Samihan Yedkar, Onur Mutlu, and Can Alkan. Accelerating Read Mapping with FastHASH. *BMC Genomics*, 2013.
- [87] Hongyi Xin, John Greth, John Emmons, Gennady Pekhimenko, Carl Kingsford, Can Alkan, and Onur Mutlu. Shifted Hamming Distance: A Fast and Accurate SIMD-friendly Filter to Accelerate Alignment Verification in Read Mapping. *Bioinformatics*, 2015.
- [88] Max Doblas, Po Jui Shih, Oscar Lostes-Cazorla, Miquel Moreto, Christopher Batten,

- and Santiago Marco-Sola. Smx: Heterogeneous architecture for universal sequence alignment acceleration. In *MICRO*, 2025.
- [89] Heewoo Kim, Sanjay Sri Vallabh Singapuram, Haojie Ye, Joseph Izraelevitz, Trevor Mudge, Ronald Dreslinski, and Nishil Talati. NMP-PaK: Near-Memory Processing Acceleration of Scalable De Novo Genome Assembly. In *ISCA*, 2025.
- [90] Qian Lou, Sarath Chandra Janga, and Lei Jiang. Helix: Algorithm/architecture co-design for accelerating nanopore genome base-calling. In *PACT*, 2020.
- [91] Qian Lou and Lei Jiang. Brawl: A spintronics-based portable basecalling-in-memory architecture for nanopore genome sequencing. *IEEE CAL*, 2018.
- [92] Taha Shahroodi, Gagandeep Singh, Mahdi Zahedi, Haiyu Mao, Joel Lindegger, Can Firtina, Stephan Wong, Onur Mutlu, and Said Hamdioui. Swordfish: A Framework for Evaluating Deep Neural Network-based Basecalling using Computation-In-Memory with Non-Ideal Memristors. In *MICRO*, 2023.
- [93] Antonio Saavedra, Hans Lehnert, Cecilia Hernández, Gonzalo Carvajal, and Miguel Figueroa. Mining discriminative k-mers in DNA sequences using sketches and hardware acceleration. *IEEE Access*, 2020.
- [94] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proceedings of the VLDB Endowment*, 2020.
- [95] Arun Subramanian, Jack Wadden, Kush Goliya, Nathan Ozog, Xiao Wu, Satish Narayanasamy, David Blaauw, and Reetuparna Das. Accelerated seeding for genome sequence alignment with enumerated radix trees. In *ISCA*, 2021.
- [96] Wenqin Huangfu, Shuangchen Li, Xing Hu, and Yuan Xie. RADAR: A 3D-ReRAM based DNA Alignment Accelerator Architecture. In *DAC*, 2018.
- [97] S Karen Khatamifard, Zamshad Chowdhury, Nakul Pande, Meisam Razaviyayn, Chris H Kim, and Ulya R Karpuzcu. GeNVom: Read Mapping Near Non-Volatile Memory. *IEEE/ACM TCBB*, 2021.
- [98] Damla Senol Cali, Gurpreet S. Kalsi, Zülal Bingöl, Can Firtina, Lavanya Subramanian, Jeremie S. Kim, Rachata Ausavrungrinun, Mohammed Alser, Juan Gomez-Luna, Amirali Boroumand, Anant Norion, Allison Scibiz, Sreenivas Subramoneyon, Can Alkan, Saugata Ghose, and Onur Mutlu. GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis. In *MICRO*, 2020.
- [99] Saransh Gupta, Mohsen Imani, Behnam Khaleghi, Venkatesh Kumar, and Tajana Rosing. RAPID: A ReRAM Processing In-memory Architecture for DNA Sequence Alignment. In *ISLPED*, 2019.
- [100] Xue-Qi Li, Guang-Ming Tan, and Ning-Hui Sun. PIM-Align: A Processing-in-Memory Architecture for FM-Index Search Algorithm. *Journal of Computer Science and Technology*, 2021.
- [101] Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligns: A Processing-in-memory Accelerator for DNA Short Read Alignment Leveraging SOT-MRAM. In *DAC*, 2019.
- [102] Farzaneh Zokaei, Hamid R Zarandi, and Lei Jiang. Aligner: A Process-in-memory Architecture for Short Read Alignment in ReRAMs. *IEEE Computer Architecture Letters*, 2018.
- [103] Yatisht Turakhia, Gill Bejerano, and William J Dally. Darwin: A Genomics Co-processor Provides up to 15,000 x Acceleration on Long Read Assembly. In *ASPLOS*, 2018.
- [104] Daichi Fujiki, Arun Subramanian, Tianjun Zhang, Yu Zeng, Reetuparna Das, David Blaauw, and Satish Narayanasamy. Genax: A Genome Sequencing Accelerator. In *ISCA*, 2018.
- [105] Advait Madhavan, Timothy Sherwood, and Dmitri Strukov. Race Logic: A Hardware Acceleration for Dynamic Programming Algorithms. *ACM SIGARCH Computer Architecture News*, 2014.
- [106] HaoYu Cheng, Yong Zhang, and Yun Xu. Bitmapper2: A GPU-accelerated All-mapper Based on The Sparse Q-gram Index. *IEEE/ACM TCBB*, 2018.
- [107] Ernst Joachim Houtgast, Vlad-Mihai Sima, Koen Bertels, and Zaid Al-Ars. Hardware Acceleration of BWA-MEM Genomic Short Read Mapping for Longer Read Lengths. *Computational Biology and Chemistry*, 2018.
- [108] Ernst Joachim Houtgast, Vlad-Mihai Sima, Koen Bertels, and Zaid Al-Ars. An Efficient GPU-accelerated Implementation of Genomic Short Read Mapping with BWA-MEM. *ACM SIGARCH Computer Architecture News*, 2017.
- [109] Alberto Zeni, Giulia Guidi, Marquita Ellis, Nan Ding, Marco D Santambrogio, Steven Hofmeyr, Aydin Buluç, Leonid Oliker, and Katherine Yelick. Logan: High-performance GPU-based X-drop Long-read Alignment. In *IPDPS*, 2020.
- [110] Nauman Ahmed, Jonathan Lévy, Shanshan Ren, Hamid Mushtaq, Koen Bertels, and Zaid Al-Ars. GASAL2: A GPU Accelerated Sequence Alignment Library for High-Throughput NGS Data. *BMC Bioinformatics*, 2019.
- [111] Takahiro Nishimura, Jacir L Bordin, Yasuaki Ito, and Koji Nakano. Accelerating the Smith-Waterman Algorithm Using Bitwise Parallel Bulk Computation Technique on GPU. In *IPDPSW*, 2017.
- [112] Edans Flavius de Oliveira Sandes, Guillermo Miranda, Xavier Martorell, Eduard Ayguade, George Teodoro, and Alba Cristina Magalhães Melo. CUDAlign 4.0: Incremental Speculative Traceback for Exact Chromosome-wide Alignment in GPU Clusters. *IEEE TPDS*, 2016.
- [113] Yongchao Liu and Bertil Schmidt. GSWABE: Faster GPU-accelerated Sequence Alignment with Optimal Alignment Retrieval for Short DNA Sequences. *Concurrency and Computation: Practice and Experience*, 2015.
- [114] Yongchao Liu, Adrianto Wirawan, and Bertil Schmidt. CUDASW++ 3.0: Accelerating Smith-Waterman Protein Database Search by Coupling CPU and GPU SIMD Instructions. *BMC Bioinformatics*, 2013.
- [115] Yongchao Liu, Douglas L Maskell, and Bertil Schmidt. CUDASW++: Optimizing Smith-Waterman Sequence Database Searches for CUDA-enabled Graphics Processing Units. *BMC Research Notes*, 2009.
- [116] Yongchao Liu, Bertil Schmidt, and Douglas L Maskell. CUDASW++ 2.0: Enhanced Smith-Waterman Protein Database Search on CUDA-enabled GPUs Based on SIMD and Virtualized SIMD Abstractions. *BMC Research Notes*, 2010.
- [117] Richard Wilton, Tamas Budavari, Ben Langmead, Sarah J Wheelan, Steven L Salzberg, and Alexander S Szalay. Arioc: High-throughput Read Alignment with GPU-accelerated Exploration of The Seed-and-extend Search Space. *PeerJ*, 2015.
- [118] Amit Goyal, Hyuk Jung Kwon, Kichan Lee, Reena Garg, Seon Young Yun, Yoon Hee Kim, Sunghoon Lee, and Min Seob Lee. Ultra-fast Next Generation Human Genome Sequencing Data Processing Using DRAGEN Bio-IT Processor for Precision Medicine. *Open Journal of Genetics*, 2017.
- [119] Yu-Ting Chen, Jason Cong, Zhenman Fang, Jie Lei, and Peng Wei. When Spark Meets FPGAs: A Case Study for Next-Generation DNA Sequencing Acceleration. In *USENIX HotCloud*, 2016.
- [120] Peng Chen, Chao Wang, Xi Li, and Xuehai Zhou. Accelerating the Next Generation Long Read Mapping with the FPGA-based System. *IEEE/ACM TCBB*, 2014.
- [121] Yen-Lung Chen, Bo-Yi Chang, Chia-Hsiang Yang, and Tzi-Dar Chiueh. A High-Throughput FPGA Accelerator for Short-Read Mapping of the Whole Human Genome. *IEEE TPDS*, 2021.
- [122] Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A Genome Sequencing Accelerator for Optimal Alignments in Subminimal Space. In *MICRO*, 2020.
- [123] Subho Sankar Banerjee, Mohamed El-Hadedy, Jong Bin Lim, Zbigniew T Kalbarczyk, Deming Chen, Steven S Lumetta, and Ravishankar K Iyer. ASAP: Accelerated Short-read Alignment on Programmable Hardware. *IEEE TC*, 2019.
- [124] Xia Fei, Zou Dan, Lu Lina, Man Xin, and Zhang Chunlei. FPGASW: Accelerating Large-scale Smith-Waterman Sequence Alignment Application with Backtracking on FPGA Linear Systolic Array. *Interdisciplinary Sciences: Computational Life Sciences*, 2018.
- [125] Hasitha Muthumala Waidyasooriya and Masanori Hariyama. Hardware-acceleration of Short-read Alignment Based on the Burrows-wheeler Transform. *IEEE TPDS*, 2015.
- [126] Yu-Ting Chen, Jason Cong, Jie Lei, and Peng Wei. A Novel High-throughput Acceleration Engine for Read Alignment. In *FCCM*, 2015.
- [127] Enzo Rucci, Carlos Garcia, Guillermo Botella, Armando De Giusti, Marcelo Naiouf, and Manuel Prieto-Matias. SWIFOLD: Smith-Waterman Implementation on FPGA with OpenCL for Long DNA Sequences. *BMC Systems Biology*, 2018.
- [128] Abbas Haghi, Santiago Marco-Sola, Lluc Alvarez, Dionysios Diamantopoulos, Christoph Hagleitner, and Miquel Moreto. An FPGA Accelerator of the Wave-front Algorithm for Genomics Pairwise Alignment. In *FPL*, 2021.
- [129] Luyi Li, Jun Lin, and Zhongfeng Wang. PipeBSW: A Two-Stage Pipeline Structure for Banded Smith-Waterman Algorithm on FPGA. In *ISVLSI*, 2021.
- [130] Tae Jun Ham, David Bruns-Smith, Brendan Sweeney, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Young H Oh, Krste Asanovic, Jae W Lee, and Lisa Wu Wills. Genesis: A Hardware Acceleration Framework for Genomic Data Analysis. In *ISCA*, 2020.
- [131] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Jae W Lee, David Bruns-Smith, Brendan Sweeney, Krste Asanovic, Young H Oh, and Lisa Wu Wills. Accelerating Genomic Data Analytics With Composable Hardware Acceleration Framework. *IEEE Micro*, 2021.
- [132] Lisa Wu, David Bruns-Smith, Frank A. Nothaft, Qijing Huang, Sagar Karandikar, Johnny Le, Andrew Lin, Howard Mao, Brendan Sweeney, Krste Asanovic, David A. Patterson, and Anthony D. Joseph. FPGA Accelerated Indel Realignment in the Cloud. In *HPCA*, 2019.
- [133] Roman Kaplan, Leonid Yavits, and Ran Ginosar. BioSEAL: In-memory biological sequence alignment accelerator for large-scale genomic data. In *SYSTOR*, 2020.
- [134] Lingxi Wu, Rasool Sharifi, Marzieh Lenjani, Kevin Skadron, and Ashish Venkat. Sieve: Scalable in-situ dram-based accelerator designs for massively parallel k-mer matching. In *ISCA*, 2021.
- [135] Taha Shahroodi, Mahdi Zahedi, Abhairaj Singh, Stephan Wong, and Said Hamdioui. KrakenOnMem: a memristor-augmented HW/SW framework for taxonomic profiling. In *ICS*, 2022.
- [136] Taha Shahroodi, Mahdi Zahedi, Can Firtina, Mohammed Alser, Stephan Wong, Onur Mutlu, and Said Hamdioui. Demeter: A fast and energy-efficient food profiler using hyperdimensional computing in memory. *IEEE Access*, 2022.
- [137] Zuher Jahshan, Itay Merlin, Esteban Garzón, and Leonid Yavits. DASH-CAM: Dynamic Approximate Search Content Addressable Memory for genome classification. In *MICRO*, 2023.
- [138] Robert Hanhan, Esteban Garzón, Zuher Jahshan, Adam Teman, Marco Lanuzza, and Leonid Yavits. Edam: edit distance tolerant approximate matching content addressable memory. In *ISCA*, 2022.
- [139] Zhuowen Zou, Hanning Chen, Prathyush Poduval, Yeseong Kim, Mahdi Imani, Elaheh Sadreini, Rosario Cammarota, and Mohsen Imani. BioHD: an efficient genome sequence search platform using HyperDimensional memorization. In *ISCA*, 2022.
- [140] Fan Zhang, Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligner-D: Leveraging In-DRAM Computing to Accelerate DNA Short Read Alignment. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2023.
- [141] Melina Soysal, Konstantina Koliogeorgi, Can Firtina, Nika Mansouri Ghiasi, Rakesh Nadig, Haiyu Mao, Geraldo Francisco de Oliveira Junior, Yu Liang, Klea Zambaku, Mohammad Sadrosadati, et al. MARS: Processing-in-memory acceleration of raw signal genome analysis inside the storage subsystem. In *ICS*, 2025.
- [142] Damla Senol Cali, Konstantinos Kanellopoulos, Joel Lindegger, Zülal Bingöl, Gurpreet S Kalsi, Ziyi Zuo, Can Firtina, Meryem Banu Cavlak, Jeremie Kim, Nika Mansouri Ghiasi, et al. SeGraM: A universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping. In *ISCA*, 2022.
- [143] Jeremie S Kim, Damla Senol Cali, Hongyi Xin, Donghyuk Lee, Saugata Ghose,

- Mohammed Alser, Hasan Hassan, Oguz Ergin, Can Alkan, and Onur Mutlu. GRIM-Filter: Fast Seed Location Filtering in DNA Read Mapping Using Processing-in-memory Technologies. *BMC Genomics*, 2018.
- [144] Haiyu Mao, Mohammed Alser, Mohammad Sadrosadati, Can Firtina, Akanksha Baranwal, Damla Senol Cali, Aditya Manglik, Nour Almadhoun Alser, and Onur Mutlu. GenPIP: In-Memory Acceleration of Genome Analysis via Tight Integration of Basecalling and Read Mapping. In *MICRO*, 2022.
- [145] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alser, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. GenStore: A High-Performance in-Storage Processing System for Genome Sequence Analysis. In *ASPLoS*, 2022.
- [146] Lingxi Wu, Minxuan Zhou, Weihong Xu, Ashish Venkat, Tajana Rosing, and Kevin Skadron. Abakus: Accelerating k-mer Counting With Storage Technology. *TACO*, 2023.
- [147] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Ma, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing. In *ISCA*, 2024.
- [148] Sang-Woo Jun, Huy T. Nguyen, Vijay Gadepally, and Arvind. In-storage Embedded Accelerator for Sparse Pattern Processing. In *HPEC*, 2016.
- [149] You-Kai Zheng, Ming-Liang Wei, Hsiang-Yun Cheng, Chia-Lin Yang, Ming-Hsiang Tsai, Chia-Chun Chien, Yuan-Hao Zhong, Po-Hao Tseng, and Hsiang-Pang Li. In-Storage Read-Centric Seed Location Filtering Using 3D-NAND Flash for Genome Sequence Analysis. In *ASPAC*, 2025.
- [150] Longlong Chen, Jianfeng Zhu, Guiqiang Peng, Mingxu Liu, Shaojun Wei, and Leibo Liu. GEM: Ultra-Efficient Near-Memory Reconfigurable Acceleration for Read Mapping by Dividing and Predictive Scattering. *IEEE TPDS*, 2023.
- [151] GuiXin Guo, Shuang Qiu, ZhiQiang Ye, BingQiang Wang, Lin Fang, Mian Lu, Simon See, and Rui Mao. GPU-accelerated adaptive compression framework for genomics data. In *BigData*, 2013.
- [152] Weikang Qiao, Zhenman Fang, Mau-Chung Frank Chang, and Jason Cong. An FPGA-based BWT accelerator for Bzip2 data compression. In *FCCM*, 2019.
- [153] Baofu Zhao, Yubin Li, Yu Wang, and Huazhong Yang. Streaming sorting network based BWT acceleration on FPGA for lossless compression. In *ICFPT*, 2017.
- [154] Lei Jiang and Farzaneh Zokaee. Exma: A genomics accelerator for exact-matching. In *HPCA*, 2021.
- [155] James Arram, Moritz Pflanzner, Thomas Kaplan, and Wayne Luk. FPGA acceleration of reference-based compression for genomic data. In *FPT*, 2015.
- [156] Yuanrong Wang, Xueqi Li, Dawei Zhang, Guangming Tan, and Ninghui Sun. Accelerating FM-index search for genomic data processing. In *ICPP*, 2018.
- [157] E. Jebamalar Leavline and DAAG Singh. Hardware implementation of LZMA data compression algorithm. *International Journal of Applied Information Systems*, 2013.
- [158] Se-Min Lim, Seongyoung Kang, and Sang-Woo Jun. Bancroft: Genomics acceleration beyond on-device memory. *arXiv*, 2025.
- [159] Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. PIM-Aligner: A processing-in-MRAM platform for biological sequence alignment. In *DATE*, 2020.
- [160] Oxford Nanopore Technologies. MinION Mk1CO. <https://nanoporetech.com/document/requirements/minion-mk1c-spec>, 2024.
- [161] Aspyr Palatnick, Bin Zhou, Elodie Ghedin, and Michael C Schatz. iGenomics: Comprehensive DNA sequence analysis on your Smartphone. *GigaScience*, 2020.
- [162] Zachary S. Ballard, Calvin Brown, and Aydogan Ozcan. Mobile Technologies for the Discovery, Analysis, and Engineering of the Global Microbiome. *ACS Nano*, 2018.
- [163] Josephine B. Oehler, Helen Wright, Zornitza Stark, Andrew J. Mallett, and Ulf Schmitz. The application of long-read sequencing in clinical settings. *Human Genomics*, 2023.
- [164] Mrinalini Watsa, Gideon A. Erkenwick, Aaron Pomerantz, and Stefan Probst. Portable sequencing as a teaching tool in conservation and biodiversity research. *PLOS Biology*, 2020.
- [165] Joshua Quick, Nicholas J. Loman, Sophie Duraffour, Jared T. Simpson, Ettore Severi, Lauren Cowley, Joseph Akoi Bore, Raymond Koundouno, Gytis Dudas, Amy Mikhail, Nobila Ouédraogo, Babak Afrough, Amadou Bah, Jonathan H. J. Baum, Beate Becker-Ziaja, Jan Peter Boettcher, Mar Cabeza-Cabrerizo, Álvaro Camino-Sánchez, Lisa L. Carter, Juliane Doerrbecker, Theresa Enkirch, Isabel García Dorival, Nicole Hetzelt, Julia Hinzmann, Tobias Holm, Liana Eleni Kafetzopoulou, Michel Koropogui, Abigail Kosgey, Eeva Kuisma, Christopher H. Logue, Antonio Mazzei, Sarah Meisel, Marc Mertens, Janine Michel, Didier Ngabo, Katja Nitzsche, Elisa Pallasch, Livia Victoria Patrono, Jasmine Portmann, Johanna Gabriella Repits, Natasha Y. Rickett, Andreas Sachse, Katrin Singethan, Inês Vitoriano, Rahel L. Yemanaberhan, Elsa G. Zekeng, Trina Racine, Alexander Bello, Amadou Alpha Sall, Ousmane Faye, Oumar Faye, N'Faly Magassouba, Cecelia V. Williams, Victoria Amburgey, Linda Winona, Emily Davis, Jon Gerlach, Frank Washington, Vanessa Monteil, Marine Jourdain, Marion Bererd, Alimou Camara, Hermann Somlare, Abdoulaye Camara, Marianne Gerard, Guillaume Bado, Bernard Baillet, Déborah Delaune, Koumpingnin Yacouba Nebie, Abdoulaye Diarra, Yacouba Savane, Raymond Bernard Pallawo, Giovanna Jaramillo Gutierrez, Natacha Milhano, Isabelle Roger, Christopher J. Williams, Facinet Yattara, Kuama Lewandowski, James Taylor, Phillip Rachwal, Daniel J. Turner, Georgios Pollakis, Julian A. Hiscoc, David A. Matthews, Matthew K. O' Shea, Andrew McD. Johnston, Duncan Wilson, Emma Hutley, Erasmus Smit, Antonino Di Caro, Roman Wölfel, Kilian Stoecker, Erna Fleischmann, Martin Gabriel, Simon A. Weller, Lamine Koivogui, Boubacar Diallo, Sakoba Keita, Andrew Rambaut, Pierre Formenty, Stephan Günther, and Miles W. Carroll. Real-time, portable genome sequencing for Ebola surveillance. *Nature*, 2016.
- [166] Yunhao Wang, Yue Zhao, Audrey Bollas, Yuru Wang, and Kin Fai Au. Nanopore Sequencing Technology, Bioinformatics and Applications. *Nature Biotechnology*, 2021.
- [167] Mark Adler. pigz: A parallel implementation of gzip for modern multi-processor, multi-core machines. *Jet Propulsion Laboratory*, 2015.
- [168] Mikel Hernaez, Dmitri Pavlichin, Tsachy Weissman, and Idoia Ochoa. Genomic Data Compression. *Annual Review of Biomedical Data Science*, 2019.
- [169] ARM Holdings. Cortex-R4. <https://developer.arm.com/ip-products/processors/cortex-r/cortex-r4>, 2011.
- [170] Samsung. Samsung SSD 860 PRO. <https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/860pro/>, 2018.
- [171] Mohammed Alser, Jeremy Rotman, Dhriti Deshpande, Kody Taraszka, Huwenbo Shi, Pelin Icer Baykal, Harry Taegyun Yang, Victor Xue, Sergey Knyazev, Benjamin D. Singer, Brunilda Balliu, David Koslicki, Pavel Skums, Alex Zelikovsky, Can Alkan, Onur Mutlu, and Serghei Mangul. Technology Dictates Algorithms: Recent Developments in Read Alignment. *Genome Biology*, 2021.
- [172] Nicholas Stoler and Anton Nekrutenko. Sequencing error profiles of illumina sequencing instruments. *NAR Genomics and Bioinformatics*, 2021.
- [173] Sara Goodwin, John D McPherson, and W Richard McCombie. Coming of Age: Ten Years of Next-generation Sequencing Technologies. *Nature Reviews Genetics*, 2016.
- [174] Eric M Davis, Yu Sun, Yanling Liu, Pandurang Kolekar, Ying Shao, Karol Szelachta, Heather L Mulder, Dongren Ren, Stephen V Rice, Zhaoming Wang, et al. SequenErr: measuring and suppressing sequencer errors in next-generation sequencing data. *Genome Biology*, 2021.
- [175] Mantas Sereika, Rasmus Hansen Kirkegaard, Søren Michael Karst, Thomas Yssing Michaelsen, Emil Aarre Sørensen, Rasmus Dam Wollenberg, and Mads Albertsen. Oxford Nanopore R10.4 long-read sequencing enables the generation of near-finished bacterial genomes from pure cultures and metagenomes without short-read or reference polishing. *Nature Methods*, 2022.
- [176] Miten Jain, Sergey Koren, Karen H. Miga, Josh Quick, Arthur C. Rand, Thomas A. Sasaki, John R. Tyson, Andrew D. Beggs, Alexander T. Dilthey, Ian T. Fiddes, Sunir Malla, Hannah Marriott, Tom Nieto, Justin O'Grady, Hugh E. Olsen, Brent S. Pedersen, Arang Rhie, Hollian Richardson, Aaron R. Quinlan, Terrance P. Snutch, Louise Tee, Benedict Paten, Adam M. Phillippy, Jared T. Simpson, Nicholas J. Loman, and Matthew Loose. Nanopore Sequencing and Assembly of A Human Genome with Ultra-long Reads. *Nature Biotechnology*, 2018.
- [177] Alexander Payne, Nadine Holmes, Vardman Rakyan, and Matthew Loose. BulkVis: a graphical viewer for Oxford nanopore bulk FAST5 files. *Bioinformatics*, 2018.
- [178] Shanika L Amarasinghe, Shian Su, Xueyi Dong, Luke Zappia, Matthew E Ritchie, and Quentin Gouil. Opportunities and Challenges in Long-read Sequencing Data Analysis. *Genome Biology*, 2020.
- [179] Ting Hon, Kristin Mars, Greg Young, Yu-Chih Tsai, Joseph W. Karalius, Jane M. Landolin, Nicholas Maurer, David Kudrna, Michael A. Hardigan, Cynthia C. Steiner, Steven J. Knapp, Doreen Ware, Beth Shapiro, Paul Peluso, and David R. Rank. Highly Accurate Long-read HiFi Sequencing Data for Five Complex Genomes. *Scientific Data*, 2020.
- [180] Ying Ni, Xudong Liu, Zemenu Mengistie Simeneh, Mengsu Yang, and Runsheng Li. Benchmarking of Nanopore R10.4 and R9.4.1 flow cells in single-cell whole-genome amplification and whole-genome shotgun sequencing. *Computational and Structural Biotechnology Journal*, 2023.
- [181] Aaron M. Wenger, Paul Peluso, William J. Rowell, Pi-Chuan Chang, Richard J. Hall, Gregory T. Concepcion, Jana Ebler, Arkarachi Functammasan, Alexey Kolesnikov, Nathan D. Olson, Armin Töpfer, Michael Alonge, Medhat Mahmoud, Yufeng Qian, Chen-Shan Chin, Adam M. Phillippy, Michael C. Schatz, Gene Myers, Mark A. DePristo, Jue Ruan, Tobias Marschall, Fritz J. Sedlazeck, Justin M. Zook, Heng Li, Sergey Koren, Andrew Carroll, David R. Rank, and Michael W. Hunkapiller. Accurate Circular Consensus Long-read Sequencing Improves Variant Detection and Assembly of A Human Genome. *Nature Biotechnology*, 2019.
- [182] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, Michael L. Heuer, and Peter M. Rice. The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 2009.
- [183] Gagandeep Singh, Mohammed Alser, Kristof Denolf, Can Firtina, Alireza Khodamoradi, Meryem Banu Cavlak, Henk Corporaal, and Onur Mutlu. RUBICON: a framework for designing efficient deep learning-based genomic basecallers. *Genome Biology*, 2024.
- [184] Zhimeng Xu, Yuting Mai, Denghui Liu, Wenjun He, Xinyuan Lin, Chi Xu, Lei Zhang, Xin Meng, Joseph Mafofo, Walid Abbas Zaher, et al. Fast-bonito: a faster deep learning based basecaller for nanopore sequencing. *Artificial Intelligence in the Life Sciences*, 2021.
- [185] Hiruna Samarakoon, James M Ferguson, Hasindu Gamaarachchi, and Ira W Deveson. Accelerated nanopore basecalling with SLOW5 data format. *Bioinformatics*, 2023.
- [186] Meryem Banu Cavlak, Gagandeep Singh, Mohammed Alser, Can Firtina, Joël Lindegger, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Can Alkan, and Onur Mutlu. Targetcall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering. *APBC*, 2023.
- [187] Brent Ewing and Phil Green. Base-Calling of Automated Sequencer Traces Using Phred. II. Error Probabilities. *Genome Research*, 1998.
- [188] Illumina. Sequence file formats for a variety of data analysis options. <https://www.illumina.com/informatics/sequencing-data-analysis/sequence-file-formats.html>, 2024.
- [189] Ben Langmead and Steven L Salzberg. Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 2012.
- [190] Daehwan Kim, Joseph M Paggi, Chanhee Park, Christopher Bennett, and Steven L

- Salzberg. Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology*, 2019.
- [191] Heng Li. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv*, 2013.
- [192] Nathan D. Olson, Justin Wagner, Nathan Dwarshuis, Karen H. Miga, Fritz J. Sedlazeck, Marc Salit, and Justin M. Zook. Variant calling and benchmarking in an era of complete human genome sequences. *Nature Reviews Genetics*, 2023.
- [193] Ryan Poplin, Pi-Chuan Chang, David Alexander, Scott Schwartz, Thomas Colthurst, Alexander Ku, Dan Newburger, Jojo Dijamco, Nam Nguyen, Pegah T. Afshar, Sam S. Gross, Lizzie Dorfman, Cory Y. McLean, and Mark A. DePristo. A universal SNP and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 2018.
- [194] Heng Li, Jue Ruan, and Richard Durbin. Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Research*, 2008.
- [195] Y. William Yu, Deniz Yorukoglu, Jian Peng, and Bonnie Berger. Quality score compression improves genotyping accuracy. *Nature Biotechnology*, 2015.
- [196] Jimin Park, Daniel E. Cook, Pi-Chuan Chang, Alexey Kolesnikov, Lucas Brambrink, Juan Carlos Mier, Joshua Gardner, Brandy McNulty, Samuel Sacco, Ayse G. Keskus, Asher Bryant, Tanveer Ahmad, Jyoti Shetty, Yongmei Zhao, Bao Tran, Giuseppe Narzisi, Adrienne Helland, Byunggil Yoo, Irina Pushel, Lisa A. Lansdon, Chengpeng Bi, Adam Walter, Margaret Gibson, Tomi Pastinen, Rebecca Reiman, Sharvari Mankame, T. Rhyker Ranallo-Benavidez, Christine Brown, Nicolas Robine, Floris P. Barthel, Midhat S. Farooqi, Karen H. Miga, Andrew Carroll, Mikhail Kolmogorov, Benedict Paten, and Kishwar Shafin. Accurate somatic small variant discovery for multiple sequencing technologies with DeepSomatic. *Nature Biotechnology*, 2025.
- [197] Can Firtina, Jeremie S Kim, Mohammed Alser, Damla Senol Cali, A Erçument Cicek, Can Alkan, and Onur Mutlu. Apollo: A Sequencing-technology-independent, Scalable and Accurate Assembly Polishing Algorithm. *Bioinformatics*, 2020.
- [198] Damla Senol Cali, Jeremie S Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Nanopore Sequencing Technology and Tools for Genome Assembly: Computational Analysis of the Current State, Bottlenecks and Future Directions. *Briefings in Bioinformatics*, 2018.
- [199] Heng Li. Minimap and Miniasm: Fast Mapping and De Novo Assembly for Noisy Long Sequences. *Bioinformatics*, 2016.
- [200] Ehsan Haghsheenas, Hossein Asghari, Jens Stoye, Cedric Chauve, and Faraz Hach. HASLR: Fast Hybrid Assembly of Long Reads. *iScience*, 2020.
- [201] Aleksey V. Zimin, Daniela Puiu, Ming-Cheng Luo, Tingting Zhu, Sergey Koren, Guillaume Marçais, James A. Yorke, Jan Dvořák, and Steven L. Salzberg. Hybrid assembly of the large and highly repetitive genome of *Aegilops tauschii*, a progenitor of bread wheat, with the MaSuRCA mega-reads algorithm. *Genome Research*, 2017.
- [202] Derrick E Wood, Jennifer Lu, and Ben Langmead. Improved Metagenomic Analysis with Kraken 2. *Genome Biology*, 2019.
- [203] Derrick E Wood and Steven L Salzberg. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 2014.
- [204] Duy Tin Truong, Eric A Franzosa, Timothy L Tickle, Matthias Scholz, George Weingart, Edoardo Pasolli, Adrian Tett, Curtis Huttenhower, and Nicola Segata. MetaPhlAn2 for Enhanced Metagenomic Taxonomic Profiling. *Nature Methods*, 2015.
- [205] Daehwan Kim, Li Song, Florian P Breitwieser, and Steven L Salzberg. Centrifuge: Rapid and Sensitive Classification of Metagenomic Sequences. *Genome Research*, 2016.
- [206] Li Song and Ben Langmead. Centrifuge: lossless compression of microbial genomes for efficient and accurate metagenomic sequence classification. *Genome Biology*, 2024.
- [207] Rachid Ounit, Steve Wanamaker, Timothy J Close, and Stefano Lonardi. CLARK: Fast and Accurate Classification of Metagenomic and Genomic Sequences Using Discriminative K-mers. *BMC Genomics*, 2015.
- [208] Vitor C Piro, Martin S Lindner, and Bernhard Y Renard. DUDes: a top-down taxonomic profiler for metagenomics. *Bioinformatics*, 2016.
- [209] Jeremy Fan, Steven Huang, and Samuel D Chortlon. BugSeq: a highly accurate cloud platform for long-read metagenomic analyses. *BMC Bioinformatics*, 2021.
- [210] Vitor C Piro, Temesgen H Dadi, Enrico Seiler, Knut Reinert, and Bernhard Y Renard. gano: precise metagenomics classification against large and up-to-date sets of reference sequences. *Bioinformatics*, 2020.
- [211] Vanessa R Marcelino, Philip TLC Clausen, Jan P Buchmann, Michelle Wille, Jonathan R Iredell, Wieland Meyer, Ole Lund, Tania C Sorrell, and Edward C Holmes. CCMetagen: comprehensive and accurate identification of eukaryotes and prokaryotes in metagenomic data. *Genome Biology*, 2020.
- [212] Alessio Milanese, Daniel R. Mende, Lucas Paoli, Guillem Salazar, Hans-Joachim Ruscheweyh, Miguelangel Cuenca, Pascal Hingamp, Renato Alves, Paul I. Costea, Luis Pedro Coelho, Thomas S. B. Schmidt, Alexandre Almeida, Alex L. Mitchell, Robert D. Finn, Jaime Huerta-Cepas, Peer Bork, Georg Zeller, and Shinichi Sunagawa. Microbial abundance, activity and population genomic profiling with mOTUs2. *Nature Communications*, 2019.
- [213] Nathan LaPierre, Mohammed Alser, Eleazar Eskin, David Koslicki, and Sergei Mangu. Metalign: Efficient Alignment-based Metagenomic Profiling Via Containment Min Hash. *Genome Biology*, 2020.
- [214] Christopher Pockrandt, Aleksey V. Zimin, and Steven L. Salzberg. Metagenomic classification with KrakenUni on low-memory computers. *Journal of Open Source Software*, 2022.
- [215] Alexander T Dilthey, Chirag Jain, Sergey Koren, and Adam M Phillippy. Strain-level metagenomic assignment and compositional estimation for long reads with MetaMaps. *Nature Communications*, 2019.
- [216] Téó Lemane, Nolan Lezcoche, Julien Lecubin, Eric Pelletier, Magali Lescot, Rayan Chikhi, and Pierre Peterlongo. Indexing and real-time user-friendly queries in terabyte-sized complex genomic datasets with kindex and ORA. *Nature Computational Science*, 2024.
- [217] Wei Shen, Hongyan Xiang, Tianquan Huang, Hui Tang, Mingli Peng, Dachuan Cai, Peng Hu, and Hong Ren. KMCP: accurate metagenomic profiling of both prokaryotic and viral populations by pseudo-mapping. *Bioinformatics*, 2022.
- [218] Zheng Sun, Shi Huang, Meng Zhang, Qiyun Zhu, Niina Haiminen, Anna Paola Carrieri, Yoshiki Vázquez-Baeza, Laxmi Parida, Ho-Cheol Kim, Rob Knight, and Yang-Yu Liu. Challenges in benchmarking metagenomic profilers. *Nature Methods*, 2021.
- [219] Jennifer Lu, Florian P Breitwieser, Peter Thielen, and Steven L Salzberg. Bracken: Estimating Species Abundance in Metagenomics Data. *PeerJ Computer Science*, 2017.
- [220] David Koslicki and Daniel Falush. MetaPalette: a k-mer Painting Approach for Metagenomic Taxonomic Profiling and Quantification of Novel Strain Variation. *mSystems*, 2016.
- [221] Evangelos A. Dimopoulos, Alberto Carmagnini, Irina M. Velsko, Christina Warinner, Greger Larson, Laurent A. F. Frantz, and Evan K. Irving-Pease. HAYSTAC: A Bayesian framework for robust and rapid species identification in high-throughput sequencing data. *PLOS Computational Biology*, 2022.
- [222] Fernando Meyer, Adrian Fritz, Zhi-Luo Deng, David Koslicki, Till Robin Lesker, Alexey Gurevich, Gary Robertson, Mohammed Alser, Dmitry Antipov, Francesco Beghini, Denis Bertrand, Jaqueline J. Brito, C. Titus Brown, Jan Buchmann, Aydin Buluç, Bo Chen, Rayan Chikhi, Philip T. L. C. Clausen, Alexandru Cristian, Piotr Wojciech Dabrowski, Aaron E. Darling, Rob Egan, Eleazar Eskin, Evangelos Georganas, Eugene Goltsman, Melissa A. Gray, Lars Hestbjerg Hansen, Steven Hofmeyer, Pingqin Huang, Luiz Irber, Huijue Jia, Tue Sparholt Jørgensen, Silas D. Kieser, Terje Klemetsen, Axel Kola, Mikhail Kolmogorov, Anton Korobeynikov, Jason Kwan, Nathan LaPierre, Claire Lemaitre, Chenhao Li, Antoine Limasset, Fabio Malcher-Miranda, Sergei Mangu, Vanessa R. Marcelino, Camille Marchet, Pierre Marjion, Dmitry Meleshko, Daniel R. Mende, Alessio Milanese, Niranjana Nagarajan, Jakob Nissen, Sergey Nurk, Leonid Olikier, Lucas Paoli, Pierre Peterlongo, Vitor C. Piro, Jacob S. Porter, Simon Rasmussen, Evan R. Rees, Knut Reinert, Bernhard Renard, Espen Mikal Robertsen, Gail L. Rosen, Hans-Joachim Ruscheweyh, Varuni Sarwal, Nicola Segata, Enrico Seiler, Lizhen Shi, Fengzhu Sun, Shinichi Sunagawa, Søren Johannes Sørensen, Ashleigh Thomas, Chengxuan Tong, Mirko Trajkovski, Julien Tremblay, Gherman Uritskiy, Riccardo Vicedomini, Zhengyang Wang, Ziye Wang, Zhong Wang, Andrew Warren, Nils Peder Willassen, Katherine Yelick, Ronghui You, Georg Zeller, Zhengqiao Zhao, Shanfeng Zhu, Jie Zhu, Ruben Garrido-Oter, Petra Gastmeier, Stéphane Hacquard, Susanne Häußler, Ariane Khaledi, Friederike Maechler, Fantin Mesny, Simona Radutoiu, Paul Schulze-Lefert, Nathiana Smit, Till Strowig, Andreas Bremges, Alexander Sczyrba, and Alice Carolyn McHardy. Critical Assessment of Metagenome Interpretation: the second round of challenges. *Nature Methods*, 2022.
- [223] Md Vasimuddin, Sanchit Misra, Heng Li, and Srinivas Aluru. Efficient Architecture-aware Acceleration of BWA-MEM for Multicore Systems. In *IPDPS*, 2019.
- [224] Mikhail Kolmogorov, Jeffrey Yuan, Yu Lin, and Pavel A Pevzner. Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 2019.
- [225] Haoyu Cheng, Gregory T. Concepcion, Xiaowen Feng, Haowen Zhang, and Heng Li. Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 2021.
- [226] Sergey Aganezov, Stephanie M. Yan, Daniela C. Soto, Melanie Kirsche, Samantha Zarate, Pavel Avdeyev, Dylan J. Taylor, Kishwar Shafin, Alaina Shumate, Chunlin Xiao, Justin Wagner, Jennifer McDaniel, Nathan D. Olson, Michael E. G. Sauria, Mitchell R. Vollger, Arang Rhie, Melissa Meredith, Skylar Martin, Joyce Lee, Sergey Koren, Jeffrey A. Rosenfeld, Benedict Paten, Ryan Layer, Chen-Shan Chin, Fritz J. Sedlazeck, Nancy F. Hansen, Danny E. Miller, Adam M. Phillippy, Karen H. Miga, Rajiv C. McCoy, Megan Y. Dennis, Justin M. Zook, and Michael C. Schatz. A complete reference genome improves analysis of human genetic variation. *Science*, 2022.
- [227] Jouni Sirén, Parsa Eskandar, Matteo Tommaso Ungaro, Glenn Hickey, Jordan M. Eizenga, Adam M. Novak, Xian Chang, Pi-Chuan Chang, Mikhail Kolmogorov, Andrew Carroll, Jean Monlong, and Benedict Paten. Personalized pangenome references. *Nature Methods*, 2024.
- [228] Naga Sai Kavya Vaddadi, Taher Mun, and Benjamin Langmead. Minimizing Reference Bias: The Impute-First Approach for Personalized Genome Analysis. In *BCB*, 2023.
- [229] Michael M Khayat, Sayed Mohammad Ebrahim Sahraeian, Samantha Zarate, Andrew Carroll, Huixiao Hong, Bohu Pan, Leming Shi, Richard A Gibbs, Marghoob Mohiyuddin, Yuanqing Zheng, and Fritz J Sedlazeck. Hidden Biases in Germline Structural Variant Detection. *Genome Biology*, 2021.
- [230] Stephan Köstlbacher, Astrid Collingro, Tamara Halter, Frederik Schulz, Sean P Jungbluth, and Matthias Horn. Pangenomics Reveals Alternative Environmental Lifestyles among Chlamydiae. *Nature Communications*, 2021.
- [231] Lucy van Dorp, Mislav Acman, Damien Richard, Liam P. Shaw, Charlotte E. Ford, Louise Ormond, Christopher J. Owen, Juanita Pang, Cedric S. Tan, Florencia A.T. Boshier, Arturo Torres Ortiz, and François Balloux. Emergence of genomic diversity and recurrent mutations in SARS-CoV-2. *Infection, Genetics and Evolution*, 2020.
- [232] Glennis A. Logsdon, Peter Ebert, Peter A. Audano, Mark Loftus, David Porubsky, Jana Ebler, Feyza Yilmaz, Pille Hallast, Timofey Prodanov, DongAhn Yoo, Carolyn A. Paisie, William T. Harvey, Xuefang Zhao, Gianni V. Martino, Mir Henglin, Katherine M. Munson, Keon Rabbani, Chen-Shan Chin, Bida Gu, Hufsa Ashraf, Stephan Scholz, Olanrewaju Austine-Oromoloye, Parithi Balachandran, Marc Jan Bonder, Haoyu Cheng, Zechen Chong, Jonathan Crabtree, Mark Gerstein, Lisbeth A. Guethlein, Patrick Hasenfeld, Glenn Hickey, Kendra Hoekzema, Sarah E. Hunt, Matthew Jensen, Yunzhe Jiang, Sergey Koren, Youngjun Kwon, Chong Li,

- Heng Li, Jiaqi Li, Paul J. Norman, Keisuke K. Oshima, Benedict Paten, Adam M. Phillippy, Nicholas R. Pollock, Tobias Rausch, Mikko Rautiainen, Yuwei Song, Arda Söylev, Arvis Sulovari, Likhitha Surapaneni, Vasiliki Tsalpou, Weichen Zhou, Ying Zhou, Qihui Zhu, Michael C. Zody, Ryan E. Mills, Scott E. Devine, Xinghua Shi, Michael E. Talkowski, Mark J. P. Chaisson, Alexander T. Dilthey, Miriam K. Konkel, Jan O. Korbel, Charles Lee, Christine R. Beck, Evan E. Eichler, and Tobias Marschall. Complex genetic variation in nearly complete human genomes. *Nature*, 2025.
- [233] Xiangqun Zheng-Bradley, Ian Streeter, Susan Fairley, David Richardson, Laura Clarke, Paul Flicek, and the 1000 Genomes Project Consortium. Alignment of 1000 Genomes Project reads to reference assembly GRCh38. *GigaScience*, 2017.
- [234] Nae-Chyun Chen, Luis F. Paulin, Fritz J. Sedlazeck, Sergey Koren, Adam M. Phillippy, and Ben Langmead. Improved sequence mapping using a complete reference genome and lift-over. *Nature Methods*, 2024.
- [235] Arang Rhie, Sergey Nurk, Monika Cechova, Savannah J. Hoyt, Dylan J. Taylor, Nicolas Altemose, Paul W. Hook, Sergey Koren, Mikko Rautiainen, Ivan A. Alexandrov, Jamie Allen, Mobin Asri, Andrey V. Bzikadze, Nae-Chyun Chen, Chen-Shan Chin, Mark Diekhans, Paul Flicek, Giulio Formenti, Arkarachai Functammasan, Carlos Garcia Giron, Erik Garrison, Ariel Gershman, Jennifer L. Gerton, Patrick G. S. Grady, Andrea Guarracino, Leanne Haggerty, Reza Halabian, Nancy F. Hansen, Robert Harris, Gabrielle A. Hartley, William T. Harvey, Marina Haukness, Jakob Heinz, Thibaut Hourlier, Robert M. Hubley, Sarah E. Hunt, Stephen Hwang, Miten Jain, Rupesh K. Kesharwani, Alexandra P. Lewis, Heng Li, Glennis A. Logsdon, Julian K. Lucas, Wojciech Makalowski, Christopher Markovic, Fergal J. Martin, Ann M. Mc Cartney, Rajiv C. McCoy, Jennifer McDaniel, Brandy M. McNulty, Paul Medvedev, Alla Mikheenko, Katherine M. Munson, Terence D. Murphy, Hugh E. Olsen, Nathan D. Olson, Luis F. Paulin, David Porubsky, Tamara Potapova, Fedor Ryabov, Steven L. Salzberg, Michael E. G. Sauria, Fritz J. Sedlazeck, Kishwar Shafin, Valery A. Shepelev, Alaina Shumate, Jessica M. Storer, Likhitha Surapaneni, Angela M. Taravella Oill, Françoise Thibaud-Nissen, Winston Timp, Marta Tomaszewicz, Mitchell R. Vollger, Brian P. Walenz, Allison C. Watwood, Matthias H. Weissensteiner, Aaron M. Wenger, Melissa A. Wilson, Samantha Zarate, Yiming Zhu, Justin M. Zook, Evan E. Eichler, Rachel J. O'Neill, Michael C. Schatz, Karen H. Miga, Kateryna D. Makova, and Adam M. Phillippy. The complete sequence of a human Y chromosome. *Nature*, 2023.
- [236] Sergey Nurk, Sergey Koren, Arang Rhie, Mikko Rautiainen, Andrey V. Bzikadze, Alla Mikheenko, Mitchell R. Vollger, Nicolas Altemose, Lev Uralsky, Ariel Gershman, Sergey Aganev, Savannah J. Hoyt, Mark Diekhans, Glennis A. Logsdon, Michael Alonge, Stylianos E. Antonarakis, Matthew Borchers, Gerard G. Bouffard, Shelsea Y. Brooks, Gina V. Caldas, Nae-Chyun Chen, Haoyu Cheng, Chen-Shan Chin, William Chow, Leonardo G. de Lima, Philip C. Dishuck, Richard Durbin, Tatiana Dvorkina, Ian T. Fiddes, Giulio Formenti, Robert S. Fulton, Arkarachai Functammasan, Erik Garrison, Patrick G. S. Grady, Tina A. Graves-Lindsay, Ira M. Hall, Nancy F. Hansen, Gabrielle A. Hartley, Marina Haukness, Kerstin Howe, Michael W. Hunkapiller, Chirag Jain, Miten Jain, Erich D. Jarvis, Peter Kerpeldjiev, Melanie Kirsche, Mikhail Kolmogorov, Jonas Korlak, Milinn Kremitzki, Heng Li, Valerie V. Maduro, Tobias Marschall, Ann M. McCartney, Jennifer McDaniel, Danny E. Miller, James C. Mullikin, Eugene W. Myers, Nathan D. Olson, Benedict Paten, Paul Peluso, Pavel A. Pevzner, David Porubsky, Tamara Potapova, Evgeny I. Rogae, Jeffrey A. Rosenfeld, Steven L. Salzberg, Valerie A. Schneider, Fritz J. Sedlazeck, Kishwar Shafin, Colin J. Shew, Alaina Shumate, Ying Sims, Arian F. A. Smit, Daniela C. Soto, Ivan Sović, Jessica M. Storer, Aaron Streets, Beth A. Sullivan, Françoise Thibaud-Nissen, James Torrance, Justin Wagner, Brian P. Walenz, Aaron Wenger, Jonathan M. D. Wood, Chunlin Xiao, Stephanie M. Yan, Alice C. Young, Samantha Zarate, Urvashi Surti, Rajiv C. McCoy, Megan Y. Dennis, Ivan A. Alexandrov, Jennifer L. Gerton, Rachel J. O'Neill, Winston Timp, Justin M. Zook, Michael C. Schatz, Evan E. Eichler, Karen H. Miga, and Adam M. Phillippy. The complete sequence of a human genome. *Science*, 2022.
- [237] Jeremie S Kim, Can Firtina, Meryem Banu Cavlak, Damla Senol Cali, Nastaran Hajinazar, Mohammed Alser, Can Alkan, and Onur Mutlu. AirLift: a fast and comprehensive technique for remapping alignments between reference genomes. *IEEE/ACM TCCB*, 2024.
- [238] Peng Jia, Liming Xuan, Lei Liu, and Chaochun Wei. MetaBinG: Using GPUs to accelerate metagenomic sequence classification. *PLOS One*, 2011.
- [239] Robin Kobus, André Müller, Daniel Jünger, Christian Hundt, and Bertil Schmidt. MetaCache-GPU: ultra-fast metagenomic classification. In *ICPP*, 2021.
- [240] Xuebin Wang, Taifu Wang, Zhihao Xie, Youjin Zhang, Shiqiang Xia, Ruixue Sun, Xinqiu He, Ruizhi Xiang, Qiwen Zheng, Zhencheng Liu, Jin'An Wang, Honglong Wu, Xiangqian Jin, Weijun Chen, Dongfang Li, and Zengquan He. GPMeta: a GPU-accelerated method for ultrarapid pathogen identification from metagenomic sequences. *Briefings in Bioinformatics*, 2023.
- [241] Robin Kobus, Christian Hundt, André Müller, and Bertil Schmidt. Accelerating Metagenomic Read Classification on CUDA-enabled GPUs. *BMC Bioinformatics*, 2017.
- [242] Xiaoquan Su, Xuetao Wang, Gongchao Jing, and Kang Ning. GPU-Meta-Storms: computing the structure similarities among massive amount of microbial community samples using GPU. *Bioinformatics*, 2014.
- [243] Masahiro Yano, Hiroshi Mori, Yutaka Akiyama, Takuji Yamada, and Ken Kurokawa. CLAST: CUDA implemented large-scale alignment search tool. *BMC Bioinformatics*, 2014.
- [244] Hasindu Gamaarachchi, Chun Wai Lam, Gihan Jayatilaka, Hiruna Samarakoon, Jared T Simpson, Martin A Smith, and Sri Parameswaran. GPU accelerated adaptive banded event alignment for rapid comparative nanopore signal analysis. *BMC Bioinformatics*, 2020.
- [245] Tianqi Zhang, Antonio González, Niema Moshiri, Rob Knight, and Tajana Rosing. GenoMiX: Accelerated Simultaneous Analysis of Human Genomics, Microbiome Metagenomics, and Viral Sequences. In *BioCAS*, 2023.
- [246] Gustavo Henrique Cervi, Cecilia Dias Flores, and Claudia Elizabeth Thompson. Metagenomic Analysis: A Pathway Toward Efficiency Using High-Performance Computing. In *ICICT*, 2022.
- [247] Po Jui Shih, Hassaan Saadat, Sri Parameswaran, and Hasindu Gamaarachchi. Efficient real-time selective genome sequencing on resource-constrained devices. *GigaScience*, 2023.
- [248] Kisaru Liyanage, Hiruna Samarakoon, Sri Parameswaran, and Hasindu Gamaarachchi. Efficient end-to-end long-read sequence mapping using minimap2-fpga integrated with hardware-accelerated chaining. *Scientific Reports*, 2023.
- [249] Can Firtina, Kamlesh Pillai, Gurpreet S. Kalsi, Bharathwaj Suresh, Damla Senol Cali, Jeremie S. Kim, Taha Shahroodi, Meryem Banu Cavlak, Joël Lindegger, Mohammed Alser, Juan Gómez Luna, Sreenivas Subramoney, and Onur Mutlu. ApHMM: Accelerating Profile Hidden Markov Models for Fast and Energy-efficient Genome Analysis. *TACO*, 2024.
- [250] Mohammed Alser, Hasan Hassan, Hongyi Xin, Oğuz Ergin, Onur Mutlu, and Can Alkan. GateKeeper: A New Hardware Architecture for Accelerating Pre-alignment in DNA Short Read Mapping. *Bioinformatics*, 2017.
- [251] Mohammed Alser, Hasan Hassan, Akash Kumar, Onur Mutlu, and Can Alkan. Shouji: A Fast and Efficient Pre-alignment Filter for Sequence Alignment. *Bioinformatics*, 2019.
- [252] Mohammed Alser, Onur Mutlu, and Can Alkan. MAGNET: Understanding and Improving the Accuracy of Genome Pre-alignment Filtering. *IPSI TIR*, 2017.
- [253] Mohammed Alser, Taha Shahroodi, Juan Gómez-Luna, Can Alkan, and Onur Mutlu. SneakySnake: A Fast and Accurate Universal Genome Pre-alignment Filter for CPUs, GPUs and FPGAs. *Bioinformatics*, 2020.
- [254] Zülal Bingöl, Mohammed Alser, Onur Mutlu, Ozcan Ozturk, and Can Alkan. GateKeeper-GPU: Fast and Accurate Pre-Alignment Filtering in Short Read Mapping. In *IPDPSW*, 2021.
- [255] Hongyi Xin, Sunny Nahar, Richard Zhu, John Emmons, Gennady Pekhimenko, Carl Kingsford, Can Alkan, and Onur Mutlu. Optimal Seed Solver: Optimizing Seed Selection in Read Mapping. *Bioinformatics*, 2016.
- [256] Kris A. Wetterstrand. DNA Sequencing Costs: Data from the NHGRI Genome Sequencing Program (GSP). <https://www.genome.gov/sequencingcostsdata>, 2024.
- [257] João Pedro de Magalhães, Cyril Laguerre, and Robi Tacutu. Integrative genomics of aging. In *Handbook of the Biology of Aging*. Ninth edition, 2021.
- [258] Alexander G. Bick, Ginger A. Metcalf, Kelsey R. Mayo, Lee Lichtenstein, Shimon Rura, Robert J. Carroll, Anjene Musick, Jodell E. Linder, I. King Jordan, Shashwat Deepali Nagar, Shivam Sharma, Robert Meller, Melissa Basford, Eric Boerwinkle, Mine S. Cicek, Kimberly F. Doherty, Evan E. Eichler, Stacey Gabriel, Richard A. Gibbs, David Glazer, Paul A. Harris, Gail P. Jarvik, Anthony Philippakis, Heidi L. Rehm, Dan M. Roden, Stephen N. Thibodeau, Scott Topper, Ashley L. Blegen, Samantha J. Wirkus, Victoria A. Wagner, Jeffrey G. Meyer, Donna M. Muzny, Eric Venner, Michelle Z. Mawhinney, Sean M. L. Griffith, Elvin Hsu, Hua Ling, Marcia K. Adams, Kimberly Walker, Jianhong Hu, Harsha Doddapaneni, Christie L. Kovar, Mullai Murugan, Shannon Dugan, Ziad Khan, Niall J. Lennon, Christina Austin-Tse, Eric Banks, Michael Katzen, Namrata Gupta, Emma Henricks, Katie Larsson, Sheli McDonough, Steven M. Harrison, Christopher Kachulis, Matthew S. Lebo, Cynthia L. Neben, Marcie Steeves, Alicia Y. Zhou, Joshua D. Smith, Christian D. Frazer, Colleen P. Davis, Karynne E. Patterson, Marsha M. Wheeler, Sean McGee, Christina M. Lockwood, Brian H. Shirts, Colin C. Pritchard, Mitzi L. Murray, Valeria Vasta, Dru Leistriz, Matthew A. Richardson, Jillian G. Buchan, Aparna Radhakrishnan, Niklas Krumm, Brenna W. Ehm, Sophie Schwartz, M. Morgan T. Aster, Kristian Cibulskis, Andrea Haessly, Rebecca Asch, Aurora Cremer, Kyle Degatano, Akum Shergill, Laura D. Gauthier, Samuel K. Lee, Aaron Hatcher, George B. Grant, Genevieve R. Brandt, Miguel Covarrubias, Ashley Able, Ashley E. Green, Jennifer Zhang, Henry R. Condon, Yuanyuan Wang, Moira K. Dillon, C. H. Albach, Wail Baalawi, Seung Hoan Choi, Xin Wang, Elisabeth A. Rosenthal, Andrea H. Ramirez, Sokny Lim, Siddhartha Nambiar, Bradley Ozenberger, Anastasia L. Wise, Chris Lunt, Geoffrey S. Ginsburg, Joshua C. Denny, All of Us Research Program Genomics Investigators, Manuscript Writing Group, All of Us Research Program Genomics Principal Investigators, Mayo Biobank, Genome Center: Baylor-Hopkins Clinical Genome Center, Color Genome Center: Broad, Laboratory for Molecular Medicine, Mass General Brigham, Genome Center: University of Washington, Data, Research Center, All of Us Research Demonstration Project Teams, and NIH All of Us Research Program Staff. Genomic data in the All of Us Research Program. *Nature*, 2024.
- [259] Keren Carss, Bjarni V. Halldorsson, Liping Hou, Jimmy Liu, Eleanor Wheeler, Yancy Lo, Kousik Kundu, Zhuoyi Huang, Ben Lacey, Ryan S. Dhindsa, Diana Rajan, Jelena Randjelovic, Neil Marriott, Carol E. Scott, Ahmet Sinan Yavuz, Ian Johnston, Trevor Howe, Mary Helen Black, Kari Stefansson, Robert Scott, Slavé Petrovski, Shuwei Li, Adrian Cortes, Fengyuan Hu, Quanli Wang, Oliver S. Burden, Sri V. V. Deevi, Carolina Haefliger, Kieren Lythgow, Peter H. Maccallum, Karyn Mégy, Jonathan Mitchell, Sean O'Dell, Amanda O'Neill, Katherine R. Smith, Haeyam Taiy, Menelas Pangalos, Ruth March, Sebastian Wasilewski, Hannes P. Eggertsson, Kristjan H. S. Moore, Hannes Hauswedell, Ogmundur Eiriksson, Aron Skafastson, Nokkvi Gislason, Svanhvit Sigurjonsdottir, Magnus O. Ulfarsson, Gunnar Palsson, Martein T. Hardarson, Asmundur Oddsson, Brynjar O. Jonsen, Snaedis Kristmundsdottir, Brynja D. Sigurpalsdottir, Olafur A. Stefansson, Doruk Beyter, Guillaume Holley, Vincius Tragante, Arnaldur Gylfason, Pall I. Olason, Florian Zink, Margret Aegisdottir, Sverrir T. Sverrisson, Brynjar Sigurdsson, Sigurjon A. Gudjonsson, Gunnar T. Sigurdsson, Gisli H. Halldorsson, Gardar Sveinbjornsson, Unnur Styrkarsdottir, Droplaug N. Magnúsdóttir, Steinunn Snorraddottir, Kari Kristinsson, Emilia Sobeh, Gudmar Thorleifsson, Frosti Jonsson, Pall Melsted, Ingileif Jonsdottir, Thorunn

- Rafnar, Hilma Holm, Hreinn Stefansson, Jona Saemundsdottir, Daniel F. Gudbjartsson, Olafur T. Magnusson, Gisli Masson, Unnur Thorsteinsdottir, Agnar Helgason, Hakon Jonsson, Patrick Sulem, Jatin Sandhuria, Tom G. Richardson, Laurence Howe, Chloe Robins, Dongjing Liu, Patrick Albers, Mariana Pereira, Daniel Seaton, Yury Aulchenko, John Whittaker, Manolis Dermitzakis, Toby Johnson, Jonathan Davitte, Erik Ingelsson, Julio Molineros, Yanfei Zhang, Alexander H. Li, Evan H. Baugh, Elisabeth Mlynarski, Abolfazl Doostparast Torshizi, Gamal Abdel-Azim, Brian Mautz, Karen Y. He, Jingyue Xi, Shirley Nieves-Rodriguez, Asif Khan, Songjun Xu, Xingjun Liu, Brice Sarver, Donghu Truong, Mohamed-Ramzi Temanni, Christopher D. Whelan, Letizia Goretti, Najat Khan, Belen Fraile, Tommaso Mansi, Guna Rajagopal, Shaheen Akhtar, Siobhan Austin-Guest, Robert Barber, Daniel Barrett, Tristram Bellerby, Adrian Clarke, Richard Clark, Maria Coppola, Linda Cornwell, Abby Crackett, Joseph Dawson, Callum Day, Alexander Dove, Jillian Durham, Robert Fairweather, Marcella Ferrero, Michael Fenton, Howard Fordham, Audrey Fraser, Paul Heath, Emily Heron, Gary Hornett, Lena Hughes-Hallett, David K. Jackson, Alexander Jakubowski Smith, Adam Laverack, Katharine Law, Steven R. Leonard, Kevin Lewis, Jennifer Liddle, Alice Lindsell, Sally Linsdell, Jamie Lovell, James Mack, Henry Mallalieu, Irfaan Mamun, Ana Monteiro, Leanne Morrow, Barбора Pardubska, Alexandru Popov, Lisa Sloper, Jan Squares, Ian Still, Oprah Taylor, Sam Taylor, Jaime M. Tovar Corona, Elliott Trigg, Valerie Vancollie, Paul Voak, Danni Weldon, Alan Wells, Eloise Wells, Mia Williams, Sean Wright, Nevena Miletic, Lea Lenhardt Ackovic, Marijeta Slavkovic-Ilic, Mladen Lazarevic, Louise Aigrain, Nicholas Redshaw, Michael Quail, Lesley Shirley, Scott Thurston, Peter Ellis, Laura Grout, Natalie Smerdon, Emma Gray, Richard Rance, Cordelia Langford, Rory Collins, Mark Effingham, Naomi Allen, Jonathan Sellors, Simon Sheard, Mahesh Pancholi, Caroline Clark, Lucy Burkitt-Gray, Samantha Welsh, Daniel Fry, Rachel Watson, Lauren Carson, Alan Young, Rami Mehio, Ole Schulz-Trieglaff, The UK Biobank Whole-Genome Sequencing Consortium, Manuscript Writing Group, AstraZeneca, Amgen deCode genetics, GSK, Johnson & Johnson, Sanger (Velsera Seven Bridges), U. K. Biobank, and Illumina. Whole-genome sequencing of 490,640 UK Biobank participants. *Nature*, 2025.
- [260] Erik Garrison, Andrea Guarracino, Simon Heumos, Flavia Villani, Zhigui Bao, Lorenzo Tattini, Jörg Hagmann, Sebastian Vorbrugg, Santiago Marco-Sola, Christian Kubica, David G. Ashbrook, Kaisa Thorell, Rachel L. Rusholme-Pilcher, Gianni Liti, Emilio Rudbeck, Agnieszka A. Golick, Sven Nahnsen, Zuyi Yang, Moses Njagi Mwaniki, Franklin L. Nobrega, Yi Wu, Hao Chen, Joep de Ligt, Peter H. Sudmant, Sanwen Huang, Detlef Weigel, Nicole Soranzo, Vincenza Colonna, Robert W. Williams, and Pjotr Prins. Building pangenome graphs. *Nature Methods*, 2024.
- [261] Joel Armstrong, Glenn Hickey, Mark Diekhans, Ian T. Fiddes, Adam M. Novak, Alden Deran, Qi Fang, Duo Xie, Shaohong Feng, Josefin Stiller, Diane Genereux, Jeremy Johnson, Voichita Dana Marinescu, Jessica Alföldi, Robert S. Harris, Kerstin Lindblad-Toh, David Haussler, Elinor Karlsson, Erich D. Jarvis, Guojie Zhang, and Benedict Paten. Progressive Cactus is a multiple-genome aligner for the thousand-genome era. *Nature*, 2020.
- [262] Ilia Minkin and Paul Medvedev. Scalable multiple whole-genome alignment and locally collinear block construction with Sibeliaz. *Nature Communications*, 2020.
- [263] Xiufei Chen, Haiqi Xu, Xiao Shu, and Chun-Xiao Song. Mapping epigenetic modifications by sequencing technologies. *Cell Death & Differentiation*, 2025.
- [264] Tianyuan Liu and Ana Conesa. Profiling the epigenome using long-read sequencing. *Nature Genetics*, 2025.
- [265] Brynja D. Sigurpalsdottir, Olafur A. Stefansson, Guillaume Holley, Doruk Beyter, Florian Zink, Martein P. Hardarson, Sverrir P. Sverrisson, Nina Kristinsdottir, Droplaug N. Magnusdottir, Olafur P. Magnusson, Daniel F. Gudbjartsson, Bjarni V. Halldorsson, and Kari Stefansson. A comparison of methods for detecting DNA methylation from long-read sequencing of human genomes. *Genome Biology*, 2024.
- [266] Yuan Yuan Li and Trygve O. Tollefsbol. DNA Methylation Detection: Bisulfite Genomic Sequencing Analysis. *Epigenetics Protocols*, 2011.
- [267] Mattia Forcato, Chiara Nicoletti, Koustav Pal, Carmen Maria Livi, Francesco Ferrari, and Silvio Bicciato. Comparison of computational methods for Hi-C data analysis. *Nature Methods*, 2017.
- [268] Koustav Pal, Mattia Forcato, and Francesco Ferrari. Hi-C analysis: from data generation to integration. *Biophysical Reviews*, 2019.
- [269] Erez Lieberman-Aiden, Nynke L. van Berkum, Louise Williams, Maxim Imakaev, Tobias Ragoczy, Agnes Telling, Ido Amit, Bryan R. Lajoie, Peter J. Sabo, Michael O. Dorschner, Richard Sandstrom, Bradley Bernstein, M. A. Bender, Mark Groudine, Andreas Gnirke, John Stamatoiyannopoulos, Leonid A. Mirny, Eric S. Lander, and Job Dekker. Comprehensive Mapping of Long-Range Interactions Reveals Folding Principles of the Human Genome. *Science*, 2009.
- [270] Eric W. Sayers, Jeffrey Beck, Evan E. Bolton, J. Rodney Brister, Jessica Chan, Ryan Connor, Michael Feldgarden, Anna M. Fine, Kathryn Funk, Jinna Hoffman, Sivakumar Kannan, Christopher Kelly, William Klimke, Sunghwan Kim, Stacy Lathrop, Aron Marchler-Bauer, Terence D. Murphy, Chris O'Sullivan, Erin Schmieder, Yuriy Skripchenko, Adam Stine, Françoise Thibaud-Nissen, Jiyao Wang, Jian Ye, Erin Zellers, Valerie A. Schneider, and Kim D. Pruitt. Database resources of the National Center for Biotechnology Information in 2025. *Nucleic Acids Research*, 2025.
- [271] Matthew Thakur, Alex Bateman, Cath Brooksbank, Mallory Freeberg, Melissa Harrison, Matthew Hartley, Thomas Keane, Gerard Kleywegt, Andrew Leach, Maria Levchenko, Sarah Morgan, Ellen M. McDonagh, Sandra Orchard, Irene Papatheodorou, Sameer Velankar, Juan Antonio Vizcaino, Rick Witham, Barbara Drazzil, and Johanna McEntyre. EMBL's European Bioinformatics Institute (EMBL-EBI) in 2022. *Nucleic Acids Research*, 2023.
- [272] Intel. Intel® QuickAssist Technology (Intel® QAT). <https://www.intel.com/content/www/us/en/architecture-and-technology/intel-quick-assist-technology-overview.html>, 2024.
- [273] Mai Zeng, Marcelo Lopes de Moraes, Paul W. Novak, Pearson Christopher, Ravinder Akula, and Vijayakumar Yeso. IBM zEnterprise Data Compression (zEDC): Implementation & Exploitation Use Cases. Technical report, IBM, 2020.
- [274] The Tukaani Project. XZ Utils. <https://tukaani.org/xz/>, 2024.
- [275] Haoyu Cheng, Erich D. Jarvis, Olivier Fedrigo, Klaus-Peter Koepfli, Lara Urban, Neil J. Gemmell, and Heng Li. Haplotype-resolved assembly of diploid genomes without parental data. *Nature Biotechnology*, 2022.
- [276] Léa Vandamme, Bastien Cazaux, and Antoine Limasset. K2R: Tinted de Bruijn Graphs implementation for efficient read extraction from sequencing datasets. *Bioinformatics Advances*, 2025.
- [277] Mikhail Karasikov, Harun Mustafa, Daniel Danciu, Oleksandr Kulkov, Marc Zimmermann, Christopher Barber, Gunnar Rätsch, and André Kahles. Efficient and accurate search in petabase-scale sequence repositories. *Nature*, 2025.
- [278] Rachel M. Colquhoun, Michael B. Hall, Leandro Lima, Leah W. Roberts, Kerri M. Malone, Martin Hunt, Brice Letcher, Jane Hawkey, Sophie George, Louise Pankhurst, and Zamin Iqbal. Pandora: nucleotide-resolution bacterial pan-genomics with reference graphs. *Genome Biology*, 2021.
- [279] Jarno Drost, Ruben van Boxtel, Francis Blokzijl, Tomohiro Mizutani, Nobuo Sasaki, Valentina Sasselli, Joep de Ligt, Sam Behjati, Judith E. Grolleman, Tom van Wezel, Serena Nik-Zainal, Roland P. Kuiper, Edwin Cuppen, and Hans Clevers. Use of CRISPR-modified human stem cell organoids to study the origin of mutational signatures in cancer. *Science*, 2017.
- [280] John N. Weinstein, Eric A. Collisson, Gordon B. Mills, Kenna R. Shaw, Brad A. Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M. Stuart. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 2013.
- [281] Pothuraju Rajarajeswari and Allam Apparao. DNABIT compress-genome compression algorithm. *Bioinformation*, 2011.
- [282] Bacem Saada and Jing Zhang. DNA sequence compression technique based on modified DNABIT algorithm. In *Proceedings of the World Congress on Engineering, London*, 2016.
- [283] Rakesh Nadig, Mohammad Sadrosadati, Haiyu Mao, Nika Mansouri Ghiasi, Arash Tavakkol, Jisung Park, Hamid Sarbazi-Azad, Juan Gómez Luna, and Onur Mutlu. Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses. In *ISCA*, 2023.
- [284] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, Jeremie Kim, Yixin Luo, Yaohua Wang, Nika Mansouri Ghiasi, Lois Orosa, Juan Gómez-Luna, and Onur Mutlu. FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives. In *ISCA*, 2018.
- [285] Jiho Kim, Seokwon Kang, Yongjun Park, and John Kim. Networked SSD: Flash Memory Interconnection Network for High-Bandwidth SSD. In *MICRO*, 2022.
- [286] Sungjun Cho, Beomjun Kim, Hyunuk Cho, Gyeongseob Seo, Onur Mutlu, Myung-suk Kim, and Jisung Park. AERO: Adaptive Erase Operation for Improving Lifetime and Performance of Modern NAND Flash-Based SSDs. In *ASPLOS*, 2024.
- [287] Hyeunjo Kim, Sanghun Oh, Jaeyong Lee, and Jihong Kim. Lazy Discharge: A High-Speed Energy-Efficient Read Technique for NAND Flash Memory. *IEEE Design & Test*, 2025.
- [288] Chen Zou and Andrew A. Chien. ASSASIN: Architecture Support for Stream Computing to Accelerate Computational Storage. In *MICRO*, 2022.
- [289] Fritz J. Sedlazeck, Philipp Rescheneder, Moritz Smolka, Han Fang, Maria Nattestad, Arndt von Haeseler, and Michael C. Schatz. Accurate detection of complex structural variations using single-molecule sequencing. *Nature Methods*, 2018.
- [290] Yoshinori Fukasawa, Luca Ermini, Hai Wang, Karen Carty, and Min-Sin Cheung. LongQC: a quality control tool for third generation sequencing long read data. *G3: Genes, Genomes, Genetics*, 2020.
- [291] Guillaume Bourque, Kathleen H. Burns, Mary Gehring, Vera Gorbunova, Andrei Seluanov, Molly Hammell, Michaël Imbeault, Zsuzsanna Izsvák, Henry L. Levin, Todd S. Macfarlan, Dixie L. Mager, and Cédric Feschotte. Ten things you should know about transposable elements. *Genome Biology*, 2018.
- [292] Dacheng Tian, Qiang Wang, Pengfei Zhang, Hitoshi Araki, Sihai Yang, Martin Kreitman, Thomas Nagylaki, Richard Hudson, Joy Bergelson, and Jian-Qun Chen. Single-nucleotide mutation rate increases close to insertions/deletions in eukaryotes. *Nature*, 2008.
- [293] William Amos. Variation in Heterozygosity Predicts Variation in Human Substitution Rates between Populations, Individuals and Genomic Regions. *PLOS One*, 2013.
- [294] Nathan LaPierre, Rob Egan, Wei Wang, and Zhong Wang. De novo nanopore read quality improvement using deep learning. *BMC Bioinformatics*, 2019.
- [295] Clara Delahaye and Jacques Nicolas. Sequencing DNA with nanopores: Troubles and biases. *PLOS One*, 2021.
- [296] Josie Gleeson, Adrien Leger, Yair D. J. Prawer, Tracy A. Lane, Paul J. Harrison, Wilfried Haerty, and Michael B. Clark. Accurate expression quantification from nanopore direct RNA sequencing with NanoCount. *Nucleic Acids Research*, 2021.
- [297] David A. Huffman. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 1952.
- [298] Anirban Nag, CN Ramachandra, Rajeev Balasubramanian, Ryan Stutsman, Edouard Giacomini, Hari Kambalasubramanyam, and Pierre-Emmanuel Gaillardon. GenCache: Leveraging In-cache Operators for Efficient Sequence Alignment. In *MICRO*, 2019.
- [299] Yukiteru Ono, Kiyoshi Asai, and Michiaki Hamada. PBSIM2: a simulator for long-read sequencers with a novel generative model of quality scores. *Bioinformatics*, 2020.
- [300] Anastasiya Belyaeva, Andrew Carroll, Daniel Cook, Daniel Liu, Kishwar Shafin, and Pi-Chuan Chang. Best: A Tool for Characterizing Sequencing Errors. *bioRxiv*, 2022.
- [301] Alberto Magi, Betti Giusti, and Lorenzo Tattini. Characterization of MinION nanopore data for resequencing analyses. *Briefings in Bioinformatics*, 2016.

- [302] Shilpa Garg, Arkarachai Fungtammasan, Andrew Carroll, Mike Chou, Anthony Schmitt, Xiang Zhou, Stephen Mac, Paul Peluso, Emily Hatas, Jay Ghurye, Jared Maguire, Medhat Mahmoud, Haoyu Cheng, David Heller, Justin M. Zook, Tobias Moemke, Tobias Marshall, Fritz J. Sedlazeck, John Ach, Chen-Shan Chin, George M. Church, and Heng Li. Chromosome-scale, haplotype-resolved assembly of human genomes. *Nature Biotechnology*, 2021.
- [303] Peiyong Guan and Wing-Kin Sung. Structural variation detection using next-generation sequencing data: A comparative technical review. *Methods*, 2016.
- [304] Travis C Glenn. Field Guide to Next-Generation DNA Sequencers. *Molecular Ecology Resources*, 2011.
- [305] Michael A Quail, Miriam Smith, Paul Coupland, Thomas D Otto, Simon R Harris, Thomas R Connor, Anna Bertoni, Harold P Swerdlow, and Yong Gu. A Tale of Three Next Generation Sequencing Platforms: Comparison of Ion Torrent, Pacific Biosciences and Illumina MiSeq Sequencers. *BMC Genomics*, 2012.
- [306] Mehdi Kchouk, Jean-Francois Gibrat, and Mourad Elloumi. Generations of Sequencing Technologies: from First to Next Generation. *Biology and Medicine*, 2017.
- [307] Franziska Pfeiffer, Carsten Gröber, Michael Blank, Kristian Händler, Marc Beyer, Joachim L Schultze, and Günter Mayer. Systematic Evaluation of Error Rates and Causes in Short Samples in Next-generation Sequencing. *Scientific Reports*, 2018.
- [308] Diana Crişan, Alexandru Irimia, Dan Gota, Liviu Miclea, Adela Puscasiu, Ovidiu Stan, and Honoriu Valean. Analyzing Benford's law's powerful applications in image forensics. *Applied Sciences*, 2021.
- [309] William R. Pearson. An Introduction to Sequence Similarity ("Homology") Searching. *Current Protocols in Bioinformatics*, 2013.
- [310] Amir Joudaki, Alexandru Metereze, Harun Mustafa, Ragnar Groot Koerkamp, André Kahles, and Gunnar Rättsch. Aligning distant sequences to graphs using long seed sketches. *Genome Research*, 2023.
- [311] Aparna Prasad, Eline D Lorenzen, and Michael V Westbury. Evaluating the role of reference-genome phylogenetic distance on evolutionary inference. *Molecular Ecology Resources*, 2022.
- [312] Ilya Grebnov. libbcs: A high performance data compression library, 2011.
- [313] Vince Buffalo. Quantifying the relationship between genetic diversity and population size suggests natural selection cannot explain Lewontin's Paradox. *eLife*, 2021.
- [314] 1000 Genomes Project Consortium, Adam Auton, Lisa D Brooks, Richard M Durbin, Erik P Garrison, Hyun Min Kang, Jan O Korbel, Jonathan L Marchini, Shane McCarthy, Gil A McVean, and Gonçalo R Abecasis. A Global Reference for Human Genetic Variation. *Nature*, 2015.
- [315] Myungsuk Kim, Jisung Park, Genhee Cho, Yoona Kim, Lois Orosa, Onur Mutlu, and Jihong Kim. Evanescio: Architectural Support for Efficient Data Sanitization in Modern Flash-Based Storage Systems. In *ASPLOS*, 2020.
- [316] Yu Cai, Saugata Ghose, Erich F Haratsch, Yixin Luo, and Onur Mutlu. Error Characterization, Mitigation, and Recovery in Flash-Memory-Based Solid-state Drives. In *Proc. IEEE*, 2017.
- [317] Jisung Park, Jaeyong Jeong, Sungjin Lee, Youngsun Song, and Jihong Kim. Improving Performance and Lifetime of NAND Storage Systems Using Relaxed Program Sequence. In *DAC*, 2016.
- [318] Jisung Park, Youngdon Jung, Jonghoon Won, Minji Kang, Sungjin Lee, and Jihong Kim. RansomeBlocker: a Low-Overhead Ransomware-Proof SSD. In *DAC*, 2019.
- [319] Rasko Leinonen, Hideaki Sugawara, Martin Shumway, and International Nucleotide Sequence Database Collaboration. The Sequence Read Archive. *Nucleic Acids Research*, 2010.
- [320] Guy Cochrane, Ilene Karsch-Mizrachi, Toshihisa Takagi, and International Nucleotide Sequence Database Collaboration. The International Nucleotide Sequence Database Collaboration. *Nucleic Acids Research*, 2015.
- [321] PCI-SIG. PCI Express Base Specification Revision 3.0. <https://pcisig.com/specifications>.
- [322] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Computing Surveys*, 2024.
- [323] AMD/Xilinx. Kintex Ultrascale+ FPGA (KU15P). <https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/kintex-ultrascale-plus.html%7D%7D>.
- [324] AMD. AMD® EPYC® 7742 CPU. <https://www.amd.com/en/products/cpu/amd-epyc-7742>.
- [325] Illumina. NovaSeq 6000 System Specifications. <https://emea.illumina.com/systems/sequencing-platforms/novaseq/specifications.html>, 2020.
- [326] UK10K Consortium, Klaudia Walter, Josine L Min, Jie Huang, Lucy Crooks, Yasin Memari, Shane McCarthy, John R B Perry, Changjiang Xu, Marta Futema, Daniel Lawson, Valentina Iotchkova, Stephan Schiffels, Audrey E Hendricks, Petr Danecek, Rui Li, James Floyd, Louise V Wain, Inês Barroso, Steve E Humphries, Matthew E Hurles, Eleftheria Zeggini, Jeffrey C Barrett, Vincent Plagnol, J Brent Richards, Celia M T Greenwood, Nicholas J Timpson, Richard Durbin, and Nicole Soranzo. The UK10K Project Identifies Rare Variants in Health and Disease. *Nature*, 2015.
- [327] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. Flash-Cosmos: In-Flash Bulk Bitwise Operations Using Inherent Computation Capability of NAND Flash Memory. In *MICRO*, 2022.
- [328] Synopsys, Inc. Design Compiler. <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-graphical.html>.
- [329] Global Foundries. 22nm CMOS FD-SOI technology. <https://gf.com/technology-platforms/fdx-fd-soi/>.
- [330] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A Fast and Extensible DRAM Simulator. *IEEE CAL*, 2015.
- [331] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator 1.0 Source Code. <https://github.com/CMU-SAFARI/ramulator>.
- [332] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkcı, and Onur Mutlu. Ramulator 2.0: A modern, modular, and extensible dram simulator. *IEEE CAL*, 2023.
- [333] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkcı, and Onur Mutlu. Ramulator 2.0 Source Code. <https://github.com/CMU-SAFARI/ramulator2>.
- [334] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim: A Framework for Enabling Realistic Studies of Modern Multi-queue SSD Devices. In *FAST*, 2018.
- [335] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim Source Code. <https://github.com/CMU-SAFARI/MQSim>.
- [336] Samsung. Samsung SSD PM1735. <https://www.samsung.com/semiconductor/ssd/enterprise-ssd/MZPLJ3T2HBJR-00007/>, 2020.
- [337] Samsung. Samsung SSD 870 EVO. <https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/870evo/>, 2021.
- [338] Micron Technology Inc. 4Gb: x4, x8, x16 DDR4 SDRAM Data Sheet, 2016.
- [339] Saugata Ghose, Tianshi Li, Nastaran Hajinazar, Damla Senol Cali, and Onur Mutlu. Demystifying Complex Workload-DRAM Interactions: An Experimental Study. *ACM POMACS*, 2019.
- [340] Saugata Ghose, Abdullah Giray Yaglikci, Raghav Gupta, Donghyuk Lee, Kais Kudrolli, William X. Liu, Hasan Hassan, Kevin K. Chang, Niladrish Chatterjee, Aditya Agrawal, Mike O'Connor, and Onur Mutlu. What Your DRAM Models Are Not Telling You: Lessons from a Detailed Experimental Study. *POMACS*, 2018.
- [341] Advanced Micro Devices. AMD® µProf. <https://developer.amd.com/amd-uprof/>, 2021.
- [342] Juan C. Motamayor, Keithanne Mockaitis, Jeremy Schmutz, Niina Haiminen, Donald Livingstone III, Omar Cornejo, Seth D. Findley, Ping Zheng, Filippo Uto, Stefan Royraert, Christopher Saski, Jerry Jenkins, Ram Podicheti, Meixia Zhao, Brian E. Scheffler, Joseph C. Stack, Frank A. Feltus, Guiliana M. Mustiga, Freddy Amores, Wilbert Phillips, Jean Philippe Marelli, Gregory D. May, Howard Shapiro, Jianxin Ma, Carlos D. Bustamante, Raymond J. Schnell, Dorrie Main, Don Gilbert, Laxmi Parida, and David N. Kuhn. The genome sequence of the most widely cultivated cacao type and its use to identify candidate genes regulating pod color. *Genome Biology*, 2013.
- [343] Michael A. Eberle, Epameinondas Fritzilas, Peter Krusche, Morten Källberg, Benjamin L. Moore, Mitchell A. Bekritsky, Zamin Iqbal, Han-Yu Chuang, Sean J. Humphray, Aaron L. Halpern, Semyon Kruglyak, Elliott H. Margulies, Gil McVean, and David R. Bentley. A reference data set of 5.4 million phased human variants validated by genetic inheritance from sequencing a three-generation 17-member pedigree. *Genome Research*, 2017.
- [344] Justin M. Zook, David Catoe, Jennifer McDaniel, Lindsay Vang, Noah Spies, Arend Sidow, Ziming Weng, Yuling Liu, Christopher E. Mason, Noah Alexander, Elizabeth Henaff, Alexa B.R. McIntyre, Dhruva Chandramohan, Feng Chen, Erich Jaeger, Ali Moshrefi, Khoa Pham, William Stedman, Tiffany Liang, Michael Saghbini, Zeljko Dzakula, Alex Hastie, Han Cao, Gintaras Deikus, Eric Schadt, Robert Sebra, Ali Bashir, Rebecca M. Truty, Christopher C. Chang, Natali Gulbahce, Keyan Zhao, Srinka Ghosh, Fiona Hyland, Yutao Fu, Mark Chaisson, Chunlin Xiao, Jonathan Trow, Stephen T. Sherry, Alexander W. Zaranek, Madeleine Ball, Jason Bobe, Preston Estep, George M. Church, Patrick Marks, Sofia Kyriazopoulou-Panagiotopoulou, Grace X.Y. Zheng, Michael Schnall-Levin, Heather S. Ordóñez, Patrice A. Mdivarti, Kristina Giorda, Ying Sheng, Karoline Bjarnesdatter Rypdal, and Marc Salit. Extensive Sequencing of Seven Human Genomes to Characterize Benchmark Reference Materials. *Scientific Data*, 2016.
- [345] Andrea Talenti. Sequencing Genome in a Bottle samples. <https://labs.epi2me.io/giab-2023.05/>, 2023.
- [346] Caroline Belser, Franc-Christophe Baurens, Benjamin Noel, Guillaume Martin, Corinne Cruaud, Benjamin Istace, Nabila Yahiaoui, Karine Labadie, Eva Hříbová, Jaroslav Doležel, Arnaud Lemaître, Patrick Wincker, Angélique D'Hont, and Jean-Marc Aury. Telomere-to-telomere gapless chromosomes of banana using nanopore sequencing. *Communications Biology*, 2021.
- [347] NVIDIA. NVIDIA nvCOMP. <https://docs.nvidia.com/cuda/nvcomp/>, 2024.
- [348] NVIDIA. NVIDIA A100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/a100/>, 2021.
- [349] AMD. AMD GZIP Compression & Decompression. <https://www.amd.com/en/developer/resources/alveo-apps/amd-gzip-compression-decompression.html>.
- [350] AMD. AMD Alveo™ U50 Data Center Accelerator Card. <https://www.amd.com/en/products/accelerators/alveo/u50/a-u50-p00g-pq-g.html>, 2019.
- [351] Junsun Choi. A hardware accelerator generator for zstandard decompression. Master's thesis, University of California, Berkeley, 2024.
- [352] NVIDIA. NVIDIA Tesla V100. <https://www.nvidia.com/en-gb/data-center/tesla-v100/>, 2017.
- [353] AMD. AMD Alveo™ U200 Data Center Accelerator Card (Active). <https://www.amd.com/en/products/accelerators/alveo/u200/a-u200-a64g-pq-g.html>, 2018.
- [354] Mikhail Karasikov, Harun Mustafa, Gunnar Rättsch, and André Kahles. Lossless indexing with counting de Bruijn graphs. *Genome Research*, 2022.
- [355] Gennady Pekhimenko, Evgeny Bolotin, Nandita Vijaykumar, Onur Mutlu, Todd C Mowry, and Stephen W Keckler. A Case for Toggle-Aware Compression for GPU Systems. In *HPCA*, 2016.
- [356] Gennady Pekhimenko, Evgeny Bolotin, Mike O'Connor, Onur Mutlu, Todd C Mowry, and Stephen W Keckler. Energy-Efficient Data Compression for GPU Memory Systems. In *ASPLOS*, 2015.

- [357] Gennady Pekhimenko, Vivek Seshadri, Yoongu Kim, Hongyi Xin, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch, and Todd C Mowry. Linearly Compressed Pages: A Low-Complexity, Low-Latency Main Memory Compression Framework. In *MICRO*, 2013.
- [358] Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch, and Todd C Mowry. Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches. In *PACT*, 2012.
- [359] Alper Buyuktosunoglu, David Trilla, Bulent Abali, Deanna Berger, Craig Walters, and Jang-Soo Lee. Enterprise-Class Cache Compression Design. In *HPCA*, 2024.
- [360] Magnus Ekman and Per Stenstrom. A Robust Main-Memory Compression Scheme. In *ISCA*, 2005.
- [361] Angelos Arelakis and Per Stenstrom. SC2: A Statistical Compression Cache Scheme. *ISCA*, 2014.
- [362] Nandita Vijaykumar, Gennady Pekhimenko, Adwait Jog, Abhishek Bhowmick, Rachata Ausavarungnirun, Chita Das, Mahmut Kandemir, Todd C Mowry, and Onur Mutlu. A case for Core-Assisted Bottleneck Acceleration in GPUs: Enabling Flexible Data Compression with Assist Warps. *ISCA*, 2015.